



M362 Unit 2
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



Processes

Unit **2**

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1590 3

1.1

CONTENTS

1	Introduction	5
1.1	The aims of this unit	5
2	Platforms and multitasking	6
2.1	A high-level view	6
2.2	The platform–software interface	8
3	Polling, interrupts and buffers	14
3.1	Polling	14
3.2	Interrupts	15
3.3	Polling versus interrupts	16
3.4	Buffers	16
4	The process in more detail	18
4.1	An abstract view of a process	18
4.2	Process state and operations	20
5	Scheduling policies	25
5.1	Process priorities	25
5.2	Granularity of scheduling	26
5.3	Characteristics of scheduling policies	27
5.4	Metrics	28
6	Threads and processes	33
6.1	Threads	33
6.2	The Java Virtual Machine (JVM) and operating system support for threading	34
6.3	Handling of memory in Java programs	35
7	Programming threads in Java	37
7.1	Creating threads	37
7.2	Thread states	42
7.3	Thread priorities	44
7.4	Methods controlling thread behaviour	44
7.5	The <code>InterruptedException</code>	46
7.6	GUIs and threads	47
8	Summary	48
	Glossary	50
	References	54
	Index	55

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

In *Unit 1* we indicated that it is possible to have several applications open on a computer and apparently running at the same time. In this unit we will explain how this may be achieved.

We begin by looking at the relationship between a computer's software and its hardware and at how the hardware and operating system support concurrency.

A crucial software concept in concurrency is that of a process – operating systems need to implement and use this concept effectively. To enable a user to run several applications at the same time, the operating system needs to recognise when certain events have taken place and deal with the issue of processes taking turns to execute on systems with limited CPU resources. This involves process scheduling, as well as techniques for processes to communicate with each other and the underlying hardware.

After developing an appreciation of processes and process scheduling we will consider how Java programs achieve concurrency using threads. Here we will discuss the relationship between the Java Virtual Machine (JVM) and the operating system working underneath it. Furthermore, since Java programs can have several paths of execution running through them, we will need to look at the specific classes and methods that allow us to control these threaded programs.

1.1 The aims of this unit

In this unit we aim to:

- ▶ give you an appreciation of the interaction between a program and its environment, which, in the case of Java, will include a Java Virtual Machine (JVM) (Section 2);
- ▶ explain two mechanisms by which a program can determine whether an event has occurred, known as polling and interrupts (Section 3);
- ▶ describe the concept of a process (Section 4) and some of the policies that can be used by the operating system's scheduler to deal with multiple processes (Section 5);
- ▶ explain the role of processes and threads in achieving concurrency (Section 6);
- ▶ enable you to write threaded programs in Java and use some of the methods available to control their execution (Section 7).

This unit mainly involves reading together with some exercises. There are also a small number of practical activities – some of these aim to demonstrate the issues under discussion while others will require you to carry out a small amount of programming.

2

Platforms and multitasking

In this section we will look at the relationship between programs and the platforms on which they run, starting by establishing definitions of some basic terms and recapping some of the issues raised in *Unit 1*.

We will then look, in some detail, at how the code in a program is executed and what it means when we say that the code is made to run on a platform. There are a number of layers between the hardware and a running application; here we will briefly discuss one of those layers – the operating system. This section will also provide an overview of the workings of the CPU and the different types of memory available in computer systems – some knowledge of both topics is helpful for the development of a good understanding of concurrency.

2.1 A high-level view

In *Unit 1* you were introduced to the idea of a number of activities running on a computer at the same time. Another name for an activity is a **task**, by which we mean a sequence of instructions treated as an element of work. A task may be represented by a method in a Java program for example, but the term is applied more widely and can refer to elements of work at a low (processor) or high (application) level. A running computer application might involve a number of tasks.

At the centre of the ability to execute tasks concurrently is the idea of a **process**, a characterisation of a program under execution. Each application a user is running on a computer (such as a word processor or a web browser) can be represented by a process that is running (that is, executing).

You may like to think of the program's source code as being like the script for a play and the process as being like the play in performance. The play is dynamic; it takes place over a period of time, interacting with its environment, just as a process interacts with its operating system and the installed hardware. In addition, there may be many performances of the play going on at the same time in various theatres, just as the same program may be running on many computer systems at the same time. In our illustration, the theatre represents what in computing is termed a **platform** – a particular combination of hardware and operating system.

Source code is static and forms an abstraction of the actual behaviour that takes place when a process runs and interacts with a platform. This abstraction is useful because it is easier to think about how tasks should take place at a level removed from the platform on which the code runs. Thus, the abstraction helps us to specify the tasks to be performed more quickly.

Multitasking

A platform may provide one or more processors, but it is typical that there are many more tasks to do than the platform's processor(s) can deal with at once. The term **multitasking** refers to the execution of more than one task 'at the same time'. As you saw earlier, this may mean parallel or pseudo-parallel execution.

When multiple tasks need to share processors, or other system resources, it becomes necessary to think about how these processes interact with each other and with those

resources, such as memory, a disk drive, a printer or a processor. Apart from shared resources being a performance bottleneck, it is common that only one process at a time may safely access a resource.

In this course we will not discuss the mechanics of distributing processes over various processors, but we take it as given that they may be, so that they may, or may not, run in parallel. Whenever processes can run in parallel, it is usually more efficient that they do so. Otherwise, we typically want pseudo-parallelism, in other words the appearance of executing in parallel.

Operating systems, such as UNIX or Windows XP, employ a variety of schemes to mediate between processes for access to shared resources. The job is made more complex by the potentially conflicting requirements of multitasking systems. These requirements (introduced in *Unit 1*) may be summarised as follows.

- 1 To keep hardware resources busy, so as to maximise their use. This need first arose in systems where users reserved time to run their jobs. The jobs were executed sequentially, and time when the hardware was not in use was seen as wasteful. This has become less important as hardware costs have fallen. Now most systems are dedicated to single users and spend much of their time idle.

Interestingly, a spin-off benefit is that idle systems can be put to work via distributed communications, for example, in grid computing projects such as the Africa at Home project.



Africa at Home.

- 2 To keep users productive, that is, not waiting for something to happen. This need first arose in time-sharing systems, where users interacted with a central computer in real time via terminals. The aim was to keep the system responsive and 'fair' to each user.
- 3 To mediate access to shared resources. This need arose in systems accessing shared databases, for example airline booking systems, where it is important to ensure that users see a consistent view of the data and are not allowed to corrupt the data, or interfere with another user's view of the system.

Two distinct multitasking operating system types have emerged to address these requirements.

- ▶ **Cooperative multitasking**, as seen in Windows 3.1 and MAC OS prior to OS X. Here applications have to be coded so that they allow other programs to run from time to time.
- ▶ **Pre-emptive multitasking**, present in for example Linux or Windows XP. The operating system can require one process to give way to another to allow sharing of resources (provided that the first process is not at a critical point that would make this operation risky).

Cooperative multitasking is no longer common in home computing.

If an operating system runs a process to completion before running another, it is said to be **non-pre-emptive**. This approach is common in batch processing systems in which user interaction is typically not required. (If a process runs without user interaction, it is termed a **batch process**, or a background job. For example, you might submit a document to a printer as a background job.)

Unless we state otherwise, in this unit we will assume that we are discussing *pre-emptive multitasking systems*, as this is the usual case for enterprise systems as well as for home computer users.

SAQ 1

- (a) Distinguish between a program and a process.
- (b) What are the key requirements for operating systems?

ANSWER.....

- (a) A program is static and an abstraction of the desired program behaviour. A process is the dynamic representation of a program, that is, it represents the program's execution.
- (b) There are three key requirements for operating systems:
- 1 Keeping systems busy.
 - 2 Keeping users productive (responsiveness).
 - 3 Sharing resources between users and processes.

2.2 The platform–software interface

We will now take a closer look at how a process interacts with the platform on which it is running. We briefly discuss how code is executed at the processor level and then consider how user code is executed via the **runtime environment**. The runtime environment is responsible for managing the loading and execution of a program, the storage it uses, and its interaction with software libraries and hardware, as well as its removal from memory when it finishes running.

Program execution

Before it can be executed, a program in the form of source code must be translated into a platform-dependent instruction set that the CPU can execute, called **machine code**.

- ▶ This translation can be achieved by **compilation**, in which all the source code is translated to machine code before any instructions are executed.
- ▶ Or it may be that the translation to machine code is done in steps, with translated instructions executed at each step, in which case we say that the language is **interpreted**.

A sequence of machine code instructions is executed as a process runs. The operating system must mediate between concurrently executing processes to prevent errors. One way of achieving this is for processors to run in two modes: **user mode** and **supervisor (or kernel) mode**. (In reality, processors may use more than two modes.)

In user mode a limited set of instructions can be executed; however, a processor in supervisor mode can execute another set of instructions called **privileged instructions**. These instructions are involved in such operations as performing input and output, and controlling entry and exit from supervisor mode.

Since the processor can perform privileged instructions only when it is in the right mode, control can be exercised over how privileged instructions are executed. Thus, user processes can be prevented from causing certain kinds of system failures, or arbitrarily executing sensitive code.

Code executed by users typically runs in user mode; to execute privileged code, calls are made to system libraries. If the operating system allows it, the processor can then switch to supervisor mode to carry out these actions. This approach allows policies to be enforced. For example, suppose that a user attempts to delete a file that is write-protected, or that does not belong to them. The deletion operation is a service provided by code in an operating system library, so the user code initially calls the service. The operating system can then determine whether or not the request should be allowed and, if appropriate, the operating system library code can complete the task.

SAQ 2

- (a) Why is compiled code typically not portable from one platform to another?
- (b) What is the difference between user mode and supervisor mode?
- ANSWER.....
- (a) Compilation creates platform-dependent machine code, targeted at a particular processor, therefore it is not portable.
- (b) Certain processor instructions, known as privileged instructions, cannot be executed when the processor is in user mode. User programs run in user mode and request supervisor mode instructions, where necessary, via library calls.

Memory

In *Unit 1* we referred to short-term memory (RAM) and long-term storage. In fact there are other varieties of memory, as shown in Figure 1. The fastest memory, at the top of the pyramid, is also the most expensive (so there is relatively little of it, and it is intensively used). Moving down the pyramid, the storage becomes slower, and cheaper.

Processors make use of a small amount of high-speed memory known as **registers** that have a capacity known as their **data width** (representing the size of data that can be handled at a time – the data width of a register is typically 32 or 64 bits). Registers hold data that is required immediately or that is frequently required, including instructions and data on which instructions operate.

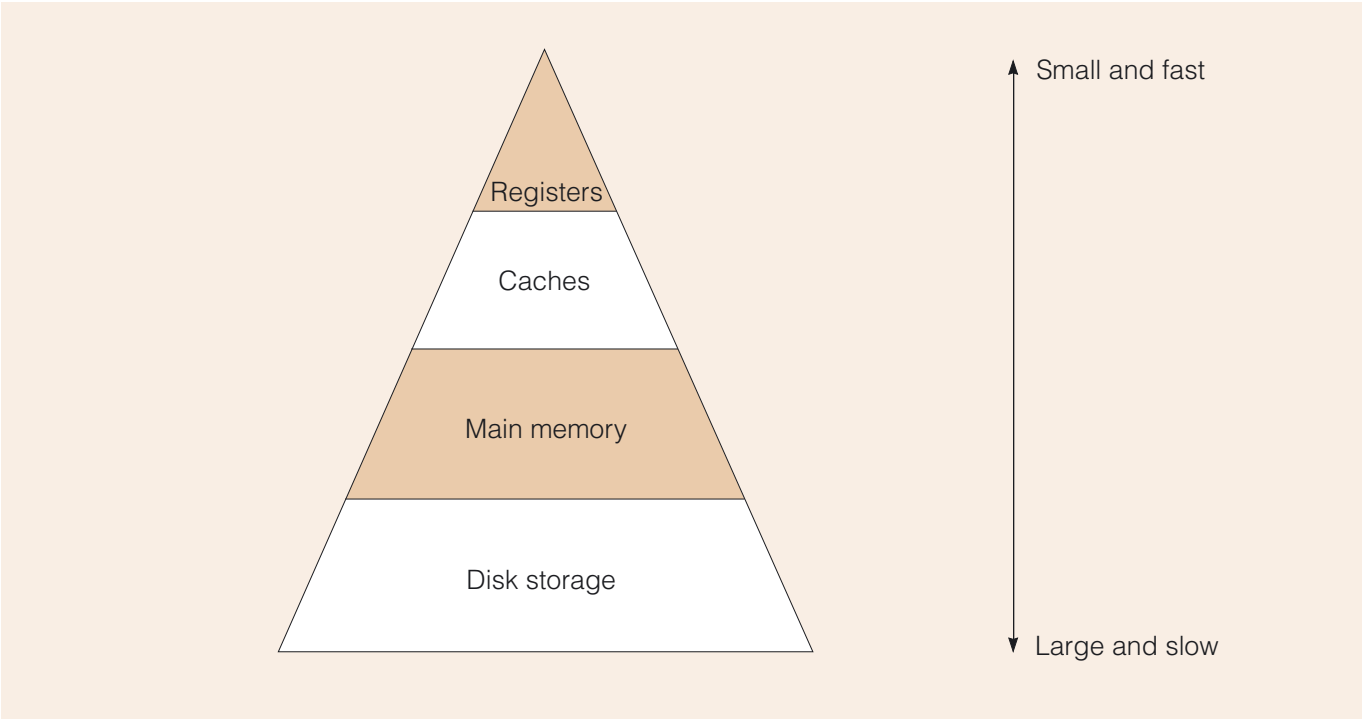


Figure 1 Memory hierarchy (also known as the memory pyramid)

Another area of high-speed, relatively local, with respect to the CPU, storage is the **cache** memory. The cache is used to hold recently accessed data or recently computed results, which are, statistically speaking, likely to be needed again soon. In addition, a system will include a relatively large amount of main memory (RAM), which provides the main storage area for processes and their data.

Disk storage can be used as so-called **virtual memory**, that is, a supplement to main memory. Using disk storage as virtual memory involves swapping information between the bottom two layers of the pyramid (main memory and disk storage) as necessary.

Main memory, caches and registers are said to be **volatile**, that is, they do not retain data when a power supply is not available, whereas disk storage is said to be **persistent** – it does retain information when power has been turned off.

Hardware interfaces

Communication between the CPU and main memory and external hardware devices such as disk drives is relatively slow, as it must take place via a 'communication highway', known as a **system bus**, with a typical data width of 64 bits. There can be delays in accessing the bus at any given time because it may already be busy.

A hardware device, such as a printer, keyboard or screen, has a **device controller**, a simple processor that provides a (hardware) interface to the device it controls. The device controller itself will have registers that hold information about the data to be transferred and help to control the transfer. Since a device controller acts independently of the main processor, the two can operate in parallel.

The relationship between these parts of a system is illustrated in Figure 2.

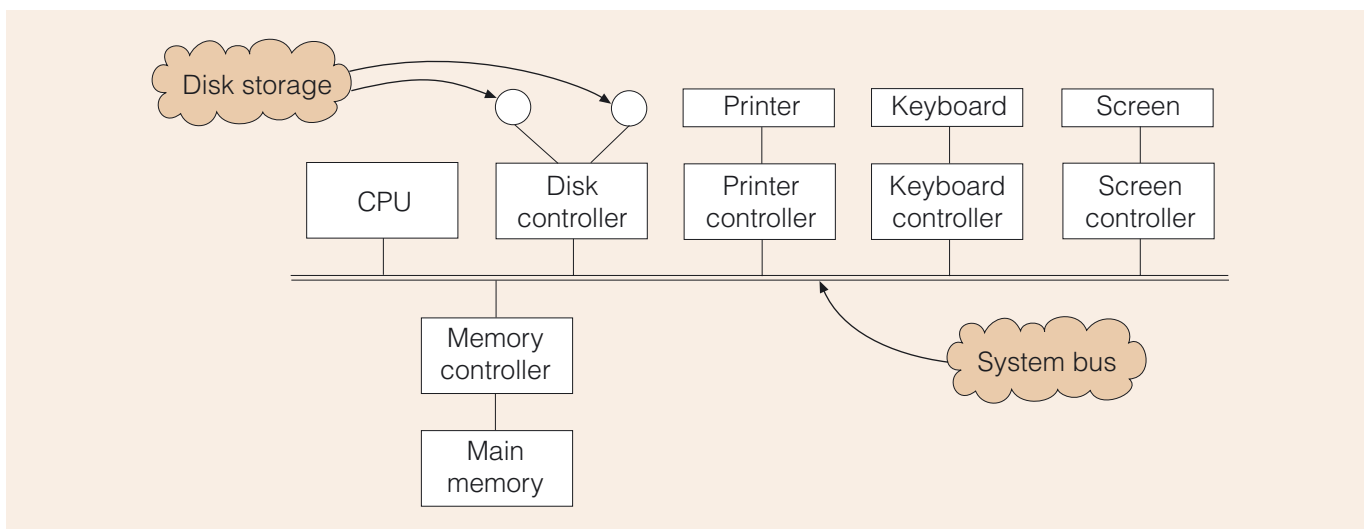


Figure 2 The architecture of a system with one processor

The processor

A CPU has three basic functions.

- 1 To write and read binary numbers to and from memory locations.
- 2 To perform arithmetic operations on numbers of a maximum size (that varies according to the processor's data width). Usually a processor makes use of at least one secondary processor dedicated to a particular task, for example, a Floating Point Unit to handle floating point arithmetic operations.
- 3 To determine what instruction should be executed next; this may involve evaluation of a condition.

Program execution takes place using the **fetch–execute cycle**. In order to make this possible, a **program counter** is used to keep track of which instruction is required next from a list of instructions in main memory. The fetch–execute cycle then consists of the following two steps, which are continually executed by the CPU.

- ▶ *Fetch* the next instruction from memory and place it in a register known as the instruction register.
- ▶ *Execute* the instruction to completion (without interruption) using registers for storage. Data may also be written to, or read from, cache memory or main memory, via the bus. The program counter is then updated.

A list of instructions is not necessarily executed in sequential order; there may be loops or jumps from one instruction to another.

The speed at which a processor can carry out its work depends on several factors, including the following:

- ▶ the size of data it can operate on;
- ▶ the data width of the bus;
- ▶ the CPU clock speed, measured in hertz (cycles per second), which determines how quickly a new instruction can be operated on;
- ▶ the speeds and sizes of the various kinds of memory the system uses.

As we are rapidly approaching physical limits in terms of processor clock speed, given current technology, parallel processing is regaining significance as a route for continued gains in computer speed.

SAQ 3

- (a) What is the purpose of the system bus?
- (b) What is the relationship between registers, main memory, disk storage, and cache?
- (c) What is a machine code instruction?

ANSWER.....

- (a) The system bus allows the processing unit to communicate with devices through their respective device controllers.
- (b) Registers are storage locations inside the CPU which hold a machine instruction and the data that it works on during a fetch–execute cycle.

A cache is similar to main memory, but is smaller and can be accessed by the CPU much faster than normal main memory. It is therefore used to hold data that is likely to be frequently accessed by the CPU.

Main memory consists of the storage locations for holding machine instructions and data that the CPU will transfer into its registers as part of the fetch–execute cycle.

Disk storage is a device attached to a computer that holds files and programs which must be loaded into main memory before they can be worked on or executed. It is a form of persistent (non-volatile) storage.

- (c) A machine code instruction is the fundamental instruction that a CPU works with and executes without interruption.
-

Software interfaces

We will now briefly consider how a user program interacts with its platform via a number of layers and interfaces between different parts of the system (see Figure 3). In so doing we will be able to trace a path from user code to hardware.

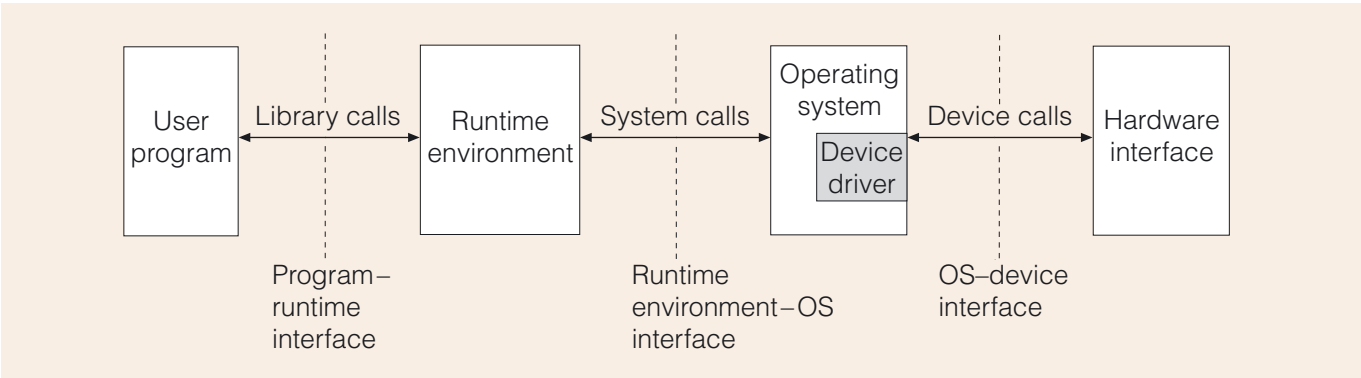


Figure 3 The interfaces between a user program and a device

Let us consider each interface in turn.

User program–runtime interface

All programs require a runtime environment to support their loading, linking them to libraries, and to manage their execution.

For example, the **Java Virtual Machine (JVM)** is a specification for a runtime environment for Java known as the **Java Runtime Environment (JRE)**. The JRE provides an environment that supports the loading and running of Java programs, allows Java programs to access Java library methods, and controls Java programs’ interaction with the operating system.

When a user program requires a system service, high-level library calls are used to interact with the runtime system.

Runtime environment–operating system interface

A runtime environment interacts with an underlying operating system through system calls to methods or procedures that are defined independently of any high-level language programs running on the system. The runtime system must make these calls on behalf of user programs. For example, a call to the `System.out.println()` method in Java will be translated into operating system-level calls, via the Java Runtime Environment.

Operating system–device interface

The operating system interacts with hardware devices (often referred to simply as devices) through the operating system–device interface (or simply device interface) by making device calls.

The operating system software includes **device drivers**, which communicate at a low level with hardware devices, using device calls, very often via a device controller incorporated in the hardware. (Device controllers have their own processors and memory.)

Note that there is also an interface between the device controller and the actual device. We will not consider this device controller–hardware interface further. For our purposes, device calls provide a software-to-hardware interface and only the device drivers need know about low-level details of device operation, such as the quantity of data transferred by a device in a single call.

Devices are very slow compared with processors, so techniques are needed to enable processors to proceed at their fastest rate. This is achieved by allowing devices and processors to operate in parallel and then, when necessary, exchange data efficiently.

The terms JVM and JRE are often used interchangeably.

SAQ 4

- (a) What are the three software interfaces that link a user program to a device controller?
- (b) How does an operating system interface to a device, and what is the role of a device driver?
- (c) What is a device controller and what is its purpose?

ANSWER.....

- (a) The three interfaces that link a user program to a device controller are:
 - ▶ the interface between the program and its runtime system (provided by library calls);
 - ▶ the interface between the runtime system and the operating system (provided by system calls);
 - ▶ the interface between the operating system and the device controller (provided by device calls).
- (b) An operating system provides a high-level interface to hardware that hides the specific details of how each device is programmed. Within the operating system, it is the device driver for a particular device that knows how to interface with the device controller and translates high-level requests into lower-level interactions with the device.
- (c) A device controller has its own simple processor, memory and registers, so that it can execute in parallel with the CPU and provide two-way communication with a hardware device. The operating system interfaces with the device controller via a device driver making device calls.

Exercise 1

If it takes 1 second to transfer 100 megabits (Mb) from a disk to main memory, and the processor executes 1000 million instructions per second, approximately how many instructions can be executed while a byte is transferred from disk to main memory?

Discussion.....

This disk drive can transfer 100 Mb in a second to memory, which is 100×2^{20} bits per second. Approximating 2^{20} as 10^6 (there is about a 5% difference), this is $100 \times 10^6 = 10^8$ bits per second. A byte is 8 bits, so this is $10^8/8$ bytes per second, or $8/10^8$ seconds to transfer one byte.

Since the processor executes 10^9 instructions per second, it will execute $10^9 \times 8/10^8$ instructions in the same time as it takes to transfer one byte. This is $10 \times 8 = 80$ instructions.

Disk manufacturers often treat 1 Mb as 10^6 bits rather than the technically correct 2^{20} bits.

3

Polling, interrupts and buffers

Sometimes a process or the operating system needs to determine whether some event has occurred before the process can continue running. An event may be, for example, data being ready from a disk drive, a process releasing a resource, or the result from a lengthy calculation being returned.

As such, there is a need for communication between processes, or between a process and a device. Here we discuss the two standard ways of arranging communication between parts of a system, called polling and interrupts.

Communicating processes or devices may not operate at the same rates, so it is often useful to provide a way of allowing each side to continue operating without having to wait for the other. This is the purpose of a buffer.

3.1 Polling

One way of determining whether an event has occurred is called **polling**, which simply involves repeatedly checking for the event. For example, you may 'poll' your wristwatch from time to time to see if it is time to go home yet. In some cases, you may have to suspend an activity while you are waiting for an event. For example, you may have to stop driving while you are waiting for a green light (although you continue running your engine and checking the traffic lights).

The delays you use between checks for events depend on factors such as the importance of the event you are looking for (for example a green light) and what else, if anything, you have to do at the time (scratch your foot, for example).

There are various policies you may apply when polling, such as checking at fixed time intervals, or at increasing time intervals, or only when you have nothing else to do. If you know roughly how long the event may take to occur, you may poll more frequently as that time approaches.

Polling may be applied in computing in order to check the status of a process, a device or a variable's value. However, it can be an inefficient technique in terms of CPU time because it may require many checks to determine whether or not an event has occurred, and you may even miss the event. Each check involves some work on the part of the processor when it might have been doing something else. For this reason, polling is also known as **busy-waiting**, meaning that you are waiting for something to happen, but you are performing work in doing so.

Suppose, for example, that your process needs to read data from a file, but the file is being written to by another process. You might like to determine whether the file is in use or not. It may be that whenever you check the file's availability the other process is busy with it, even though it has been available at moments when you were not looking. Ideally, you might like the disk drive to let you know when it is free, signalling that your process may now proceed. This is the purpose of an interrupt.

3.2 Interrupts

An **interrupt** is a signal to a processor that an event has occurred and may be generated by hardware or software. This is generally more efficient than polling and also plays an important role in helping to keep the processor doing useful work.

Typically, when the current instruction has been executed, the CPU detects any interrupt signal and the code that is currently executing is halted. Alternative code is then executed. The code that was executing can continue executing some time after the interrupt has been processed. This is known as a **context switch** and it involves pausing and resuming processes through the use of privileged instructions. We will say more about how this happens in Section 4.

An operating system can choose to ignore an interrupt signal, depending on the event itself.

An interrupt may itself be interrupted. If this happens then a decision must be taken as to whether to process the new interrupt, or delay it. For this reason, interrupts may have associated priorities, which can be used by an **interrupt handler** to determine what to do next. This may involve disabling the interrupt mechanism for a time to allow a high-priority task to be completed.

Some important classes of interrupt are as follows.

- ▶ **Program interrupts**, such as those generated by exceptions. For example, accessing an array outside of its index bounds could generate an exception, as could a program attempting to execute an illegal instruction.
- ▶ **Timer interrupts**, which allow the operating system to perform tasks at regular intervals. The timer interrupt can be used in pre-emptive multitasking systems to indicate a point at which to perform a context switch.
- ▶ **I/O interrupts**, which are generated by hardware controllers of input and output devices. For example, a mouse or keyboard in use will generate frequent I/O interrupts.
- ▶ **Faults**, which are generated due to faults with hardware.

The class of I/O interrupts is important for dealing with input and output operations with relatively slow external devices. As we have already suggested, if a processor were to have to poll for a disk to become ready, or for data to be retrieved, it would be idle much of the time. With interrupts, a process can request data and the processor can be freed to work on something else while the data is being collected. An interrupt can then indicate that the requested data is ready for collection. In the meantime the processor has not been busy-waiting, but it may have been profitably occupied in executing some other process that is not waiting for an event. When the interrupt occurs, the executing process can be paused and the process that requested the data can receive its data. Subsequently, the paused process can be restarted. In other words, context switches take place to keep the CPU busy.

An important example of the program interrupt occurs when a user process requests a resource under the control of the operating system, such as disk access, which requires execution of privileged instructions. This results in a **supervisor call interrupt**. The interrupt handler (which runs in supervisor mode) may then accept or refuse the request, for example a request to open a file that does not belong to the user may be rejected. When accepted, the interrupt results in the execution of privileged instructions. Control is later returned to the user process in user mode.

3.3 Polling versus interrupts

The use of interrupts is very general and is standard in modern computers, but many small devices operate on the basis of polling. Some device drivers support both polling and interrupt modes. For example, many wireless devices have a polling mode, which helps to save power when on the move as the device can be 'asleep' much of the time. There are also hybrid schemes, for example polling for a while and then switching to use of interrupts.

Here are some ways in which the approaches differ from each other.

Prioritisation

If several interrupts occur at the same time, they can be prioritised. Low-priority interrupts wait longer to be serviced.

Achieving prioritisation with polling would require careful consideration of the coding of any polling loops and their interaction, so is less easily done.

Efficiency

In a polling scheme it is possible to put an upper bound on the time between checking if a device needs servicing. If you have a number of devices to check, they can be tested in rotation, or some can be checked more often than others. In an interrupt-based scheme, a process may wait indefinitely for a device to signal that data is ready.

Polling is subject to a lower bound on the time between checks – sometimes it is impossible to poll at short enough intervals and interrupts must be used instead.

Interrupts are generally preferred in real-time systems where quick reaction to events is crucial. Interrupts are also preferable in interacting with users, because user-generated events are infrequent on the timescale of clock cycles. Polling in such circumstances is likely to be wasteful. Interrupts in this context also make for better responsiveness to user actions.

In general, using interrupts is more efficient than using polling as it does not involve busy-waiting, and a process can react more quickly to a signal from an external device.

Robustness

Interrupts must normally be processed, even if this only means discarding them. This can lead to a system becoming flooded with interrupts – a situation known as an **interrupt storm** – and delays in handling the interrupts can become unpredictable. This may be due to an error in a device or device driver, incorrect configuration or excessive I/O, for example.

Some operating systems switch devices between polling mode and interrupt modes according to the number of interrupts they are generating, if this is supported by the hardware. (This is also used to assist in efficient communication with devices.) Likewise, hardware may implement **interrupt rate limiting** methods to prevent excessive interrupts being generated.

3.4 Buffers

The word **buffer** is a general term for an area of memory or storage intended for temporary use. Typically there are input and output buffers on a device controller itself if the device, such as a disk drive, can perform both tasks. When input is needed *from* the device, the device places data into its output buffer (which may take some time) and, having done this, signals to the processor, using an interrupt, that the data is available to be copied into main memory. While this is going on, the processor can have executed many millions of instructions. When the processor receives the signal from the device, it

stops what it is doing and arranges for the data to be copied from the buffer into main memory. When the copying is complete, the processor can resume whatever it was doing before it was interrupted.

A similar mechanism is used for output *to* a device. Whenever the processor needs to output some data, it places the data in the device's input buffer and signals to the device that data is available. Having done this, the processor continues with other computations and leaves the device to retrieve the data. Of course, if data is already being sent to the device, it would be unwise to allow the processor to place more data into the input buffer until the previous data has been dealt with. Therefore, another signal (an interrupt) is used so that the device can indicate whether or not its buffer is ready to accept data from the processor. If not, the processor will have to wait.

The size of a buffer is determined by the need to smooth out the difference in, for example, the operating speeds of a device and processor, or even two processes.

SAQ 5

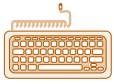
- (a) What is an interrupt? What actions must an operating system take when an interrupt occurs?
- (b) What are the advantages and disadvantages of handling devices by a polling scheme compared with an interrupt-driven approach? In what kinds of system does the application dictate the approach to be taken?
- (c) What happens if a user program attempts to execute a privileged instruction (via a system call)?
- (d) Why is it useful to have a system of priorities associated with interrupts?
- (e) What are the advantages of using a buffer for communication?

ANSWER.....

- (a) An interrupt is a mechanism by which a device or software signals to the processor that some event has occurred. If the interrupt is of high enough priority, the processor must perform a context switch. The processor must then execute a routine to deal with the interrupt (such as transfer data to a device) before resuming the original process.
 - (b) In a polling scheme every device is guaranteed to be served within a specified time, but prioritisation is difficult. In an interrupt-driven system, urgent requests can be dealt with very quickly by assigning priorities to types of request, but non-urgent requests may have to wait for a long time while more urgent requests are dealt with. Interrupt-driven systems are preferable in interactive systems and real-time systems, which require immediate responses to critical events.
 - (c) A supervisor call interrupt is generated. A context switch takes place to put the processor into supervisor mode and execute the interrupt handler. At this point the interrupt handler may decide to execute the user code or not. Subsequently the processor returns to running in user mode via another context switch.
 - (d) Prioritised interrupts enable the operating system to react more quickly to some events than others.
 - (e) A buffer can be used to smooth out differences in operating speeds of the sending and receiving processes or devices, so that communication is more efficient – given a large enough buffer neither the sending nor the receiving process (or device) will need to wait after depositing data in the buffer before proceeding with other work.
-

4

The process in more detail



Activity 2.1

What is your computer up to?

In this section we will show how a process can represent a program under execution and how this enables a computer to concurrently execute a number of applications.

A major advance in the theory behind operating systems was the idea of the representation of a process in a form that allows it (and therefore the program it represents) to be paused and resumed later. This can be done in such a way that any delay can be made negligible to the user, which is the basis of multitasking systems. Given the importance of this concept in understanding concurrency, this section will provide you with a more detailed, 'under the hood', explanation of what goes on at the level of the operating system where the processes are managed.

In this section we will discuss:

- ▶ the kind of information we need about a process to allow it to be paused and resumed;
- ▶ the various states that a process can be in from the point of view of the operating system – we will explain several models that represent these states.

4.1 An abstract view of a process

In order to pause and restart a process, we need to be able to characterise its present condition, also known as its **context**.

Operating systems make use of **context switching** to handle multiple processes – this entails storing the context of the currently running process, so that it can be restarted later, and loading that of another process so that it can be run.

The following list of items characterising a process context is not complete, but gives you some idea of the kind of information the operating system will require in order to perform context switching.

- ▶ A unique **process identifier** (a number), so that the operating system can identify each process.
- ▶ A link to the program's data (variables).
- ▶ The start address where the program instructions are located in memory.
- ▶ A program counter to indicate the next instruction to be executed.
- ▶ Register values (that is, values in use by the process when it stopped).
- ▶ A value indicating the importance of the process (its priority), which can be used to decide whether or not to run the process in preference to another.
- ▶ Events the process is waiting for, if any.
- ▶ Events the process has signalled, if any.
- ▶ How much time the process may run for.
- ▶ The process state (discussed in Subsection 4.2).

The creation of a process therefore involves creating a data structure, called a **process descriptor**, containing information such as that shown above, that can be saved and reloaded by the operating system as necessary.

When starting or restarting, a process needs to be allocated resources, particularly memory, and a process descriptor needs to be stored in some way. Process descriptors might be stored in a **process table** or some other data structure maintained by the operating system; for example, in Linux a doubly-linked list of process descriptors is maintained.

Processes will usually acquire the memory they need from a shared address space, as illustrated in Figure 4. Each process then has control of its own local memory space until it completes running or is terminated for some other reason, so that it is as if each process has its own address space.

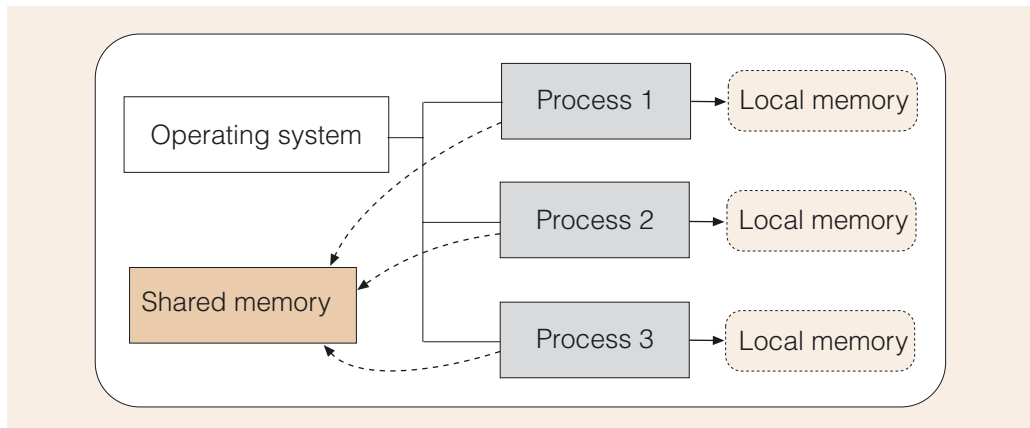


Figure 4 Allocation of memory in a multitasking system (after Oaks and Wong, 2004)

Processes may also share other resources such as external hardware devices, for example keyboards or printers. As we have already mentioned, they may also share processor time.

To return to our analogy of a play in performance, you can think of the operating system as the director of the play, while the program instructions are like the lines of the play (say, Macbeth). In a theatre with one stage (which is like having a shared processor), only one line of the play is being read at a time. If more than one play is being rehearsed, at a certain time the stage has to be cleared to allow the next play's rehearsal (perhaps Hamlet). When the time comes to give up the stage, the director will (assuming they have not finished the play) mark the place reached in the Macbeth script and note the positions of the actors and scenery – the stage can then be cleared. The stage can then be set for Hamlet, the actors take their places, and Hamlet can start or recommence from the place it left off at its last rehearsal. After another period of time, the cast of Hamlet must give way to Macbeth again; the stage must be reset, the actors take their places, and the reading of Macbeth recommences from where it left off.

To extend our analogy, a multiprocessor system could be seen as a theatre with several stages, and plays may be rehearsing exclusively on one stage or be allocated a stage when rehearsal commences (or recommences).

The management of processes is similar in many respects. Recall that a process is a program under execution and thus has various characteristics associated with a program, such as instructions to be executed, an indication of which instruction is to be executed next and associated data (variables). In this case, context switching may happen many times a second. However, there is a time overhead associated with this switching and so a balance must be struck between the time a process is allowed to run and the frequency of context switching.

In this analogy, the clearing of the stage and resetting for another play can be thought of as representing a context switch.

SAQ 6

What is a process descriptor and why is it needed?

ANSWER.....

A process descriptor is a data structure storing information about the context of a process in a form that allows the process to be paused and restarted later. Storing information about the context of a process is necessary because the system normally can execute only one process at a time. To keep the system busy and responsive, and to prevent processes from interfering with each other, we need to be able to pause and restart processes (in other words to perform context switching).

4.2 Process state and operations

A process table is used by an operating system to control the global characteristics of a system, such as whether or not all processes are being given a fair chance to execute.

Operating systems use **process schedulers** to maintain the collection of processes in the process table and determine which process is run next on a particular processor. For now, we will concentrate on how a scheduler can characterise processes to make such a choice. To assist with this, a scheduler may perform operations to place a process in a particular **process state**.

We now consider three increasingly refined models of process state, as suggested by Stallings (1991), that might allow an operating system to cater for a number of tasks at the same time.

Two-state model

The simplest model we can imagine could have just two process states, Running and Not Running. A process scheduler could have an *admit* operation to allow new processes to enter the system on creation, *timeout* and *dispatch* operations to move a process between Running and Not Running states, and a *release* operation to allow processes to exit the system when they have finished (Figure 5).

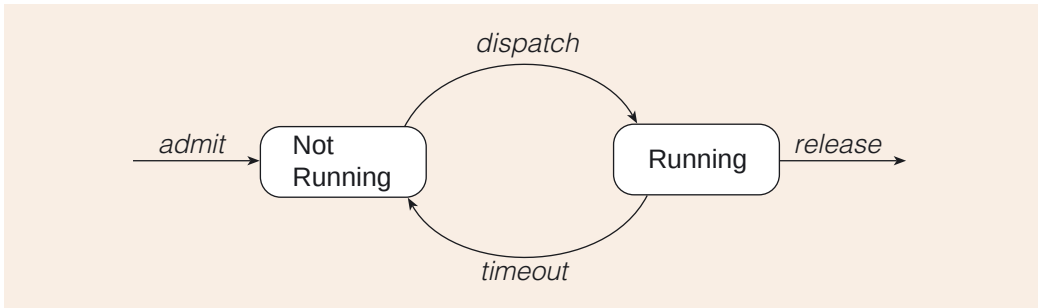


Figure 5 A two-state model for process management

Some other possible policies are mentioned in Subsection 5.3.

To implement this model, the process scheduler could maintain a queue of process identifiers, as shown in Figure 6, and then run each process as it comes to the front of the queue. A process will be run for a time-slice (also called a **quantum**), and then timed out. Its process identifier then returns to the rear of the process queue to wait its turn again.

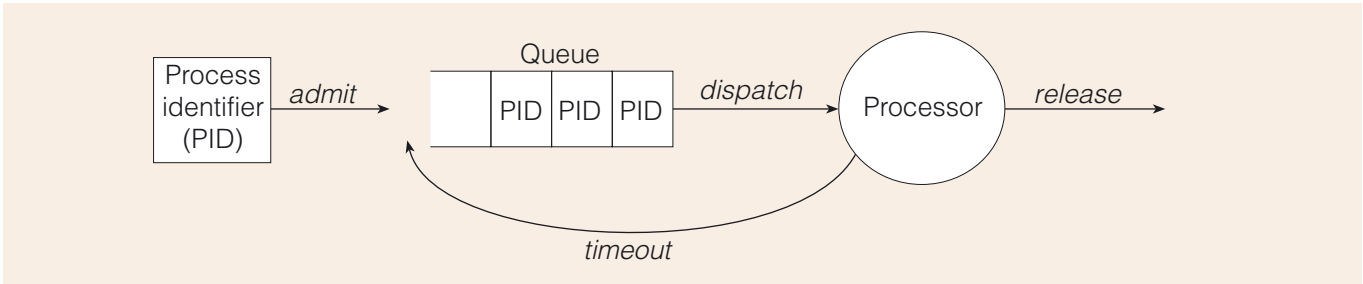


Figure 6 A queuing system for process dispatching

The two-state model is adequate provided that processes are always ready to run when called upon. However, if a process may need to wait for some event to occur, for example for a file to be ready on a disk, then there are actually two kinds of Not Running processes: ones that can proceed and ones that cannot. If a process is dispatched: but cannot run, then it must simply be paused and its identifier returned to the back of the queue, which is a waste of CPU cycles.

In order to maximise the use of the system, it is necessary for a scheduler to distinguish between processes that can proceed and those that cannot.

Five-state model

This leads us to a more complete model of process state, in which we split the Not Running state into two states, now named the Runnable and Blocked states. We have also added a New state before processes are admitted to the system, and a final Exiting state, to give us a total of five states, as shown in Figure 7.

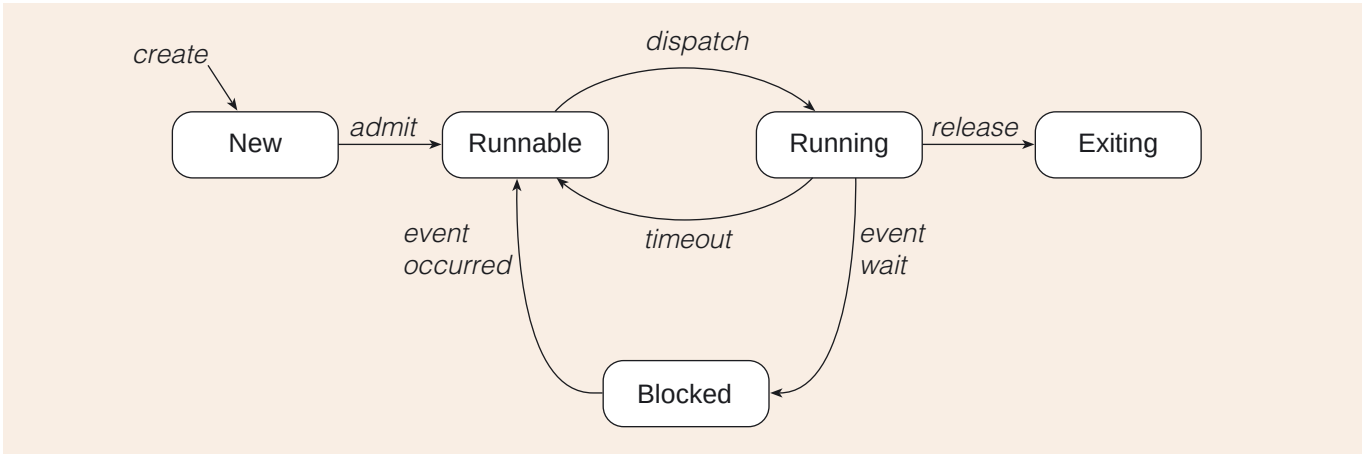


Figure 7 A five-state model for process management

In the five-state model, a process is first created by the *create* operation to place it in the New state. In the *create* operation a process descriptor is constructed, but the system has not yet included the process among those that may be run. The introduction of this state allows a system to make decisions about how many processes it can handle at any given time and which are most important. This can be used if more processes are entering the system than we have resources (chiefly main memory) to cope with.

The *admit* operation allows a process to enter the Runnable state, at which point we say the process is **runnable** or ready. The *dispatch* operation results in a process entering the Running state, as before. However, if a process cannot proceed because it requires some external event to occur, such as a resource becoming available, the *event wait* operation results in the process being **blocked** until the event takes place. The *event occurred* operation then moves the process back to a Runnable state.

A Running process may also become Runnable if it has been timed out due to the expiry of a time-slice. Finally, the extra Exiting state allows a system to perform housekeeping jobs before a process is removed from a system; for example, execution time might be recorded on a server in order to bill a client.

This approach allows a process scheduler to distinguish easily between processes that can and cannot proceed, and to execute processes that are not waiting for some event to occur.

This model may be adequate on many systems. However, because I/O is so slow compared to processor speed, it is possible that at certain points all jobs may be waiting for an event to occur, and this may still leave the processor idle much of the time. In addition, main memory will needlessly be occupied by processes that cannot proceed.

Seven-state model

This leads us to our final model of process state, which includes the idea of **swapping** a process out of main memory. A process is said to be **swapped out** when all or part of the process context is saved to external storage, for example to a disk drive. A swapped process is also said to be **suspended**. This frees resources, particularly the memory previously occupied by the swapped process, so that other processes can be admitted or dispatched.

A scheduler also needs to distinguish between those processes that have been suspended, but are no longer waiting for an event, and those that would still be blocked if they were readmitted to the system (activated). So we need to introduce two new states for processes in external memory: Blocked Suspended and Runnable Suspended. Blocked Suspended processes are waiting for an event, while Runnable Suspended processes are Runnable if they are readmitted to the system.

This model is illustrated in Figure 8.

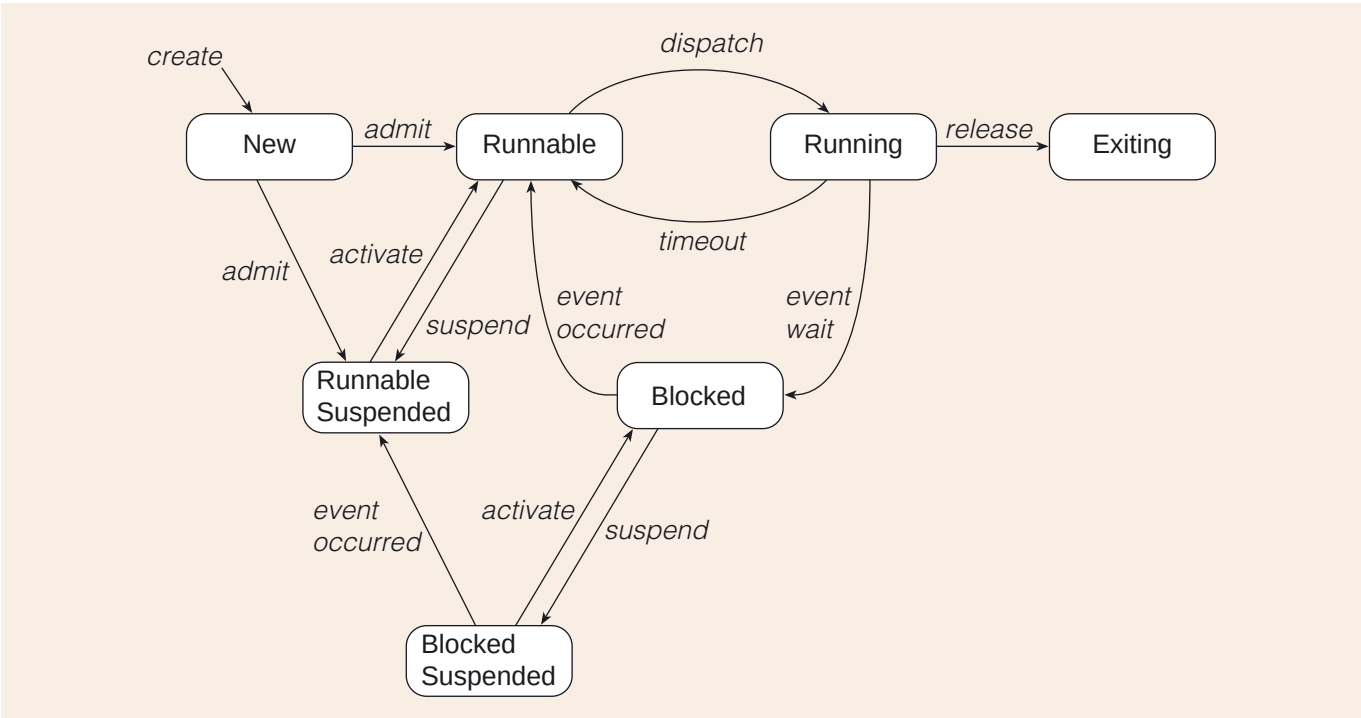


Figure 8 A seven-state model for process management

To cater for movement to one of the suspended states, we have a new *suspend* operation that can move a Blocked process to the Blocked Suspended state or a Runnable process to the Runnable Suspended state.

The *suspend* operation might be required if all of the processes that have been admitted to the system are in the Blocked state, so that New processes can be admitted (otherwise at some point the system would run out of memory). Additionally, the *suspend* operation can be used to suspend Runnable but low-priority processes, or processes requiring large amounts of memory, to allow higher-priority processes to be admitted.

The reverse transition, from external memory to main memory, takes place using a new *activate* operation. This operation may be used when there are no Runnable processes currently in main memory, or if a Runnable Suspended process is of a higher priority than those currently in main memory. Activating a Blocked Suspended process appears a bad decision, on the face of it, since the process will still be unable to proceed. However, this can be used if memory is available, the process is of a higher priority than those in the Runnable Suspended state and the operating system has some indication that the event the process is waiting for is about to occur.

In this model, the system has an extra choice for admitting processes. A New process might be made Runnable, but some process schedulers may also choose to admit from New to the Runnable Suspended state, to maintain a large pool of potentially Runnable processes. How successful this strategy might be would depend on the overhead involved in swapping processes and on the balance of Blocked versus Runnable processes at any given time.

The different process states and possible transitions between them are summarised in Table 1.

Table 1 Transition matrix for process states in the seven-state model

	New	Runnable	Running	Blocked	Blocked Suspended	Runnable Suspended	Exiting
	<i>create</i>	–	–	–	–	–	–
New	–	<i>admit</i>	–	–	–	<i>admit</i>	–
Runnable	–	–	<i>dispatch</i>	–	–	<i>suspend</i>	–
Running	–	<i>timeout</i>	–	<i>event wait</i>	–	–	<i>release</i>
Blocked	–	<i>event occurred</i>	–	–	<i>suspend</i>	–	–
Blocked Suspended	–	–	–	<i>activate</i>	–	<i>event occurred</i>	–
Runnable Suspended	–	<i>activate</i>	–	–	–	–	–
Exiting	–	–	–	–	–	–	–

In addition it is possible that processes themselves can initiate changes of state. For example, a common operation is ‘sleeping’, by which a process indicates that it will not run for a time. A process may also indicate that it will wait until it is notified of some event by another process.

SAQ 7

Under what circumstances will a process scheduler suspend a process?

ANSWER.....

Usually a process is suspended when it is Blocked and other processes might be made Runnable by being admitted, or by being activated from a Runnable Suspended state. However, a Runnable process may also be suspended according to scheduler priorities and the availability of other high-priority processes to run.

5

Scheduling policies

Context switching, together with a process state model, allows an operating system to allocate processor time to many tasks, each represented as a process in a particular state, and allows processes to be safely stopped or started as necessary.

A scheduler uses a **scheduling policy** and performs some computation using a **selection function** to decide which process should be run next.

Any policy will aim to achieve desirable characteristics such as responsiveness and high resource utilisation, but these characteristics may have varying importance on different platforms.

Selection functions may depend on parameters such as the time a process has spent waiting, the time it has spent executing, the total time it requires to complete (when known), the resources it requires, and also a measure of how important the process is, called its priority. However, the factors taken into account will vary from one policy to another.

As we have already suggested, there are two ways in which a selection function may be applied: pre-emptively and non-pre-emptively. In the pre-emptive case, a process may be required to give up the CPU even when it is logically Runnable if the process scheduler dictates this. For example, a timer interrupt can be used to signal a point at which a selection function is applied.

In the non-pre-emptive case, a process is not interrupted once started. The scheduling policy in this case does not oversee the behaviour of individual processes, but is more concerned with deciding which process to run next when the CPU is available, which it may do using priorities.

Cooperative multitasking systems are non-pre-emptive systems in which the currently running process can initiate a context switch. This means that, unless programs are written to cede control to each other, the system can easily be taken over by a single process. Also, if a process becomes hung for some reason, the whole system cannot proceed.

When a virtual machine is running, it may also implement a scheduling policy. The virtual machine in turn appears as a process within the underlying operating system and so itself may be time-sliced and scheduled in various ways.

5.1 Process priorities

A **priority** is a number that can be recorded in the process descriptor to indicate how important a process is.

Some factors that may be used to determine an initial process priority are as follows.

- ▶ The amount, or kind of, resources the process requires and the current system load on those resources.
- ▶ The importance of the user (who may have been assigned such a value on the creation of the user account). A process being run by an administrator could be assigned a high priority, while an ordinary user process could have a lower priority, for example.

- ▶ Whether the process is interactive or not. Interactive processes tend to have a higher priority than batch processes.
- ▶ Whether the process was created by a user or the operating system.
- ▶ The priority of a (parent) process responsible for creating the process in question.

The priority of a process may be changed over time; for example:

- ▶ the priority may increase when some relevant event occurs, such as the disk becoming available for a process waiting for I/O;
- ▶ process priority may be increased over time to ensure that a process is not ignored indefinitely (known as process ageing);
- ▶ process priority may decrease, for example if a process has left a user-interactive phase and entered a phase when it can perform its work in the background;
- ▶ priorities of user processes may be configurable by the user;
- ▶ processes may adjust their own priorities in reaction to changes in their requirements or progress.

Operating systems employ different numbers of priorities: for example, Solaris (a version of Unix) has 160 priority levels, while Windows NT has 5.

However, priorities are just one factor in deciding what process to run next. For example, a process with a low priority may be run if a resource it has acquired and cannot yet release is in demand from high-priority processes.

5.2 Granularity of scheduling

Although we have referred to a process scheduler, there may actually be more than one scheduler in use. Schedulers and their scheduling policies can be classified as short-, medium- or long-term.

A **short-term scheduler**, also known as a **dispatcher**, applies a short-term scheduling policy to determine transitions between the Runnable, Running and Blocked states of Figure 8, and in particular decides which of the Runnable processes to dispatch next. Depending on the policy, the dispatcher may be called into play:

- ▶ at every quantum;
- ▶ when an interrupt occurs, which may signal a change in state for some process;
- ▶ when a process changes state;
- ▶ when priorities are changed.

The processor typically executes a process even when there are no user jobs that are runnable, called the idle or **null process**. This is a way of simplifying the scheduling algorithm, but can also be used to perform routine tasks when the system is otherwise free. The null process is always runnable, and so is always in a list of runnable processes waiting to be run when no other processes are in the list.

A **medium-term scheduler** is used by systems that make use of virtual memory to swap processes in and out of main and secondary storage, according to whether they should be run immediately or may be set aside for a significant amount of time. These schedulers are concerned with policies regarding transitions to and from the Suspended states (shown in the lower part of Figure 8).

A **long-term scheduler** decides which programs are admitted to the system for execution from the New state. This may include determining how many processes are potentially runnable, what mix of processes is admitted to the system and also assessing whether response time for individual users may drop below an acceptable level by admitting another job. This will be affected by how busy the CPU or external

devices are and how heavily a program is expected to make use of these resources. These schedulers are most common in multi-user or real-time as opposed to desktop systems and, to run effectively, such schedulers must be able to deduce the resource requirements of each process.

It is usually the job of long-term and medium-term schedulers to assign priorities to processes, while a dispatcher simply picks the next process to run from those that are eligible (taking into account their priority).

A multitasking system may employ schedulers at all or some of these levels, although in practice more than one policy may be managed by the same scheduler. For example, sometimes the jobs of the medium- and long-term schedulers are performed by a single high-level scheduler.

5.3 Characteristics of scheduling policies

What makes a good policy? Here are some of the characteristics that an algorithm might attempt to optimise, the first three of which are user-oriented and the remainder of which are system-oriented performance metrics.

- ▶ **Responsiveness** – in interactive systems, this indicates whether a system is acceptably fast in its response to a request for action or information. (The definition of what is acceptable depends on the context.)
- ▶ **Turnaround** – the time a user waits for a process to complete (particularly appropriate for batch processing).
- ▶ **Deadline maximisation** – maximising the number of achieved ‘deadlines’ in real-time systems.
- ▶ **Fairness** – which requires that like processes are treated alike. This may incorporate information such as process priority, when this is available.
- ▶ **Resource utilisation** – for example keeping the CPU ‘busy’, that is, executing processes as opposed to swapping between them or waiting for I/O; not over-using resources that are in great demand.
- ▶ **Throughput** – achieving the maximum number of processes completed per unit of time.
- ▶ **Robustness** – the ability to maintain desirable characteristics under varying system loads.

There are two main sources of conflict between these objectives.

- 1 There is a conflict between achieving the best results for batch processing and keeping user interaction responsive. Processes can be seen as being on a continuum between requiring constant user interaction and no user interaction and so any policy is likely to be a compromise. As resources are used to a high degree, responsiveness goes down, while a system that can react quickly to users must have some free resources to do so.
- 2 There is a conflict between fairness and resource utilisation. Keeping the CPU busy may be seen as mostly of interest in batch processing systems, where user interaction is not required, or in expensive shared systems, where the hardware must be put to maximum use. To achieve fairness to users in interactive systems, a pre-emptive policy is generally necessary, while batch processing systems may operate with non-pre-emptive scheduling policies.

A CPU intensive process is sometimes called ‘CPU bound’.

5.4 Metrics

As you have seen, the best policy must be determined assuming a particular platform and user context.

In order to appreciate the characteristics of policies, we need to define some performance metrics.

- ▶ Let the time required to perform a job, its **service time**, be denoted s .
- ▶ Let the turnaround time, the time between a job being admitted to a system and the time it exits, be denoted q . This value may depend on the load on the system.

Although turnaround can be used to judge performance by itself, it is artificial to think that process A with a q of 100 is a hundred times worse than process B with a q of 1, because it takes no account of the service times for processes A and B . As service time increases, users are more forgiving of increases in turnaround time.

A better measure of performance is the ratio q/s , which we call a **normalised turnaround time**, as it has been scaled according to the size of the job itself. We will denote this n .

The minimum values for q and s are both 1, where we take the unit of measurement to be clock cycles. This means that the minimum value for q/s is 1, and larger values imply worse performance. The value n indicates how much longer we have to wait for a job to be done than we would have done if the job were executed straight away.

We will now use the normalised turnaround time to compare some schemes, following an example from Stallings (1991). In the examples we will assume that the processes do not require any user interaction (i.e. they are batch processes) and that the time to switch between processes is 0.

First-in, first-out (FIFO)

Perhaps the simplest policy is first-in, first-out (FIFO), a non-pre-emptive policy in which a process has to wait its turn in a queue of Runnable processes, and on reaching the front of the queue runs to completion. This is also known as ‘first-come, first-served’.

Consider a set of processes running to completion as suggested by Table 2.

Table 2 FIFO timings for some processes

Process ID	Arrival time (a)	Service time (s)	Finish time ($f = \text{sum of } s \text{ values}$)	Turnaround ($q = f - a$)	Normalised turnaround (n)
P1	0	3	3	3	1
P2	2	6	9	7	1.17
P3	4	4	13	9	2.25
P4	6	5	18	12	2.40
P5	8	2	20	12	6

The median is the middle value when items are sorted by size.

The mean value of n is about 2.56, while the median, which is another measure of average performance, is 2.25. It is also significant that the maximum value is 6.

Table 2 illustrates that the worst case under this policy is when a job with a small service time s is queued behind one or more jobs with a large (cumulative) value of s . For example, process P5 has a high value of normalised turnaround n . On average, a job with a large s will have a relatively low value of n .

The execution of these processes is illustrated in Figure 9, in which process identifiers appear on the vertical axis, and clock cycles are shown on the horizontal axis. A box indicates execution for one clock cycle, so that each process has as many boxes as its service time, s .

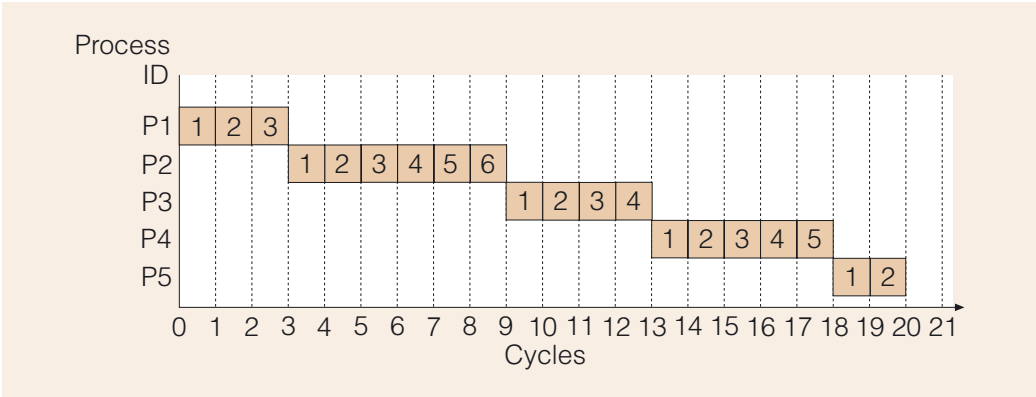


Figure 9 The FIFO execution of processes described in Table 2

Figure 9 shows that each process runs to completion according to its order of arrival, without interruption. So P1, P2, P3, P4 and then P5 are completed in that order.

This policy is not viable where frequent interaction with processes is required; however, it may appear as part of more advanced policies.

We now have data to compare with other scheduling policies. We could also have some notion of an acceptable value of n to decide whether or not to apply this policy. For example, we may decide that a value of n of 6, for P5, is too high to allow, because it says that this job took six times longer than might have been expected.

Shortest job first (SJF) scheduling

A variation on FIFO is the shortest job first scheduling policy, which is also non-pre-emptive, and is sometimes used in systems where service time can be accurately estimated. The service time may be supplied by a programmer, or may be determined from statistics obtained from previous runs.

In this approach, a queue is established from a list of Runnable processes in such a way that the job requiring least time to execute goes to the head of the queue. When any process has completed, the queue is reformed.

For the set of processes in Table 2 (that is, the same arrival and service times), we now observe the pattern shown in Figure 10.

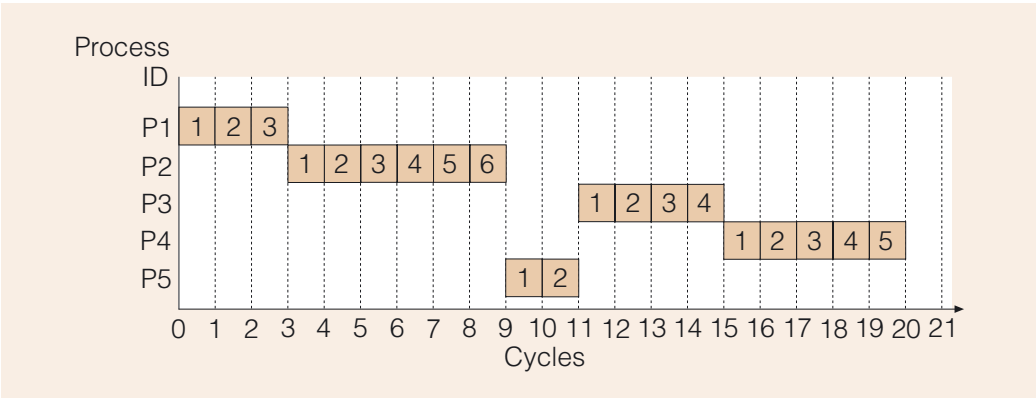


Figure 10 Shortest job first execution of processes described in Table 2

Figure 10 shows that P1 runs to completion first, in 3 steps. At this point P2 has entered the system and runs to completion in 6 steps. We are now 9 steps in and the remaining processes have all arrived. They are run to completion in order of increasing service times as P5 ($s=2$), P3 ($s=4$) and P4 ($s=5$).

The metrics for these processes are given in Table 3, for comparison with the FIFO case.

Table 3 SJF timings for some processes

Process ID	Arrival time (a)	Service time (s)	Finish time (f , as shown in Figure 10)	Turnaround ($q = f - a$)	Normalised turnaround (n)
P1	0	3	3	3	1
P2	2	6	9	7	1.17
P3	4	4	15	11	2.75
P4	6	5	20	14	2.8
P5	8	2	11	3	1.50

The mean normalised turnaround time can be calculated as 1.84, the median is 1.5 and the maximum is 2.8; all improvements on the FIFO values for n .

Throughput is also improved under this scheme, but processes with long service times must wait longer to complete, on average, than those with short service times. A disadvantage is that this policy can lead to ‘starvation’ of time-expensive processes that may be relegated to the back of the queue repeatedly.

Note also that this example related to CPU-bound jobs and the SJF scheme is not as useful when frequent interaction is required, for the same reasons as given earlier.

Round robin

Another way to improve on the FIFO policy is to introduce a time limit during which each job may hold the CPU, which produces the round robin, a pre-emptive FIFO policy.

Because this policy is pre-emptive, it is of more general use and is useful in interactive systems. This policy is very commonly applied and appears in Linux, Windows XP and the Sun JVM.

Each process is part of a queuing system for Runnable processes and is assigned a quantum during which it is allowed to run before giving way and possibly being returned to the rear of the queue. (There is no other case for pre-emption.)

We will assume that a process finishing its quantum is queued behind any newly arriving process, and behind any processes that have worked their way to the front of the queue. However, variations on this approach are possible.

Let us consider the same set of processes as before, assuming a quantum of 1 clock cycle for illustration, as shown in Figure 11.

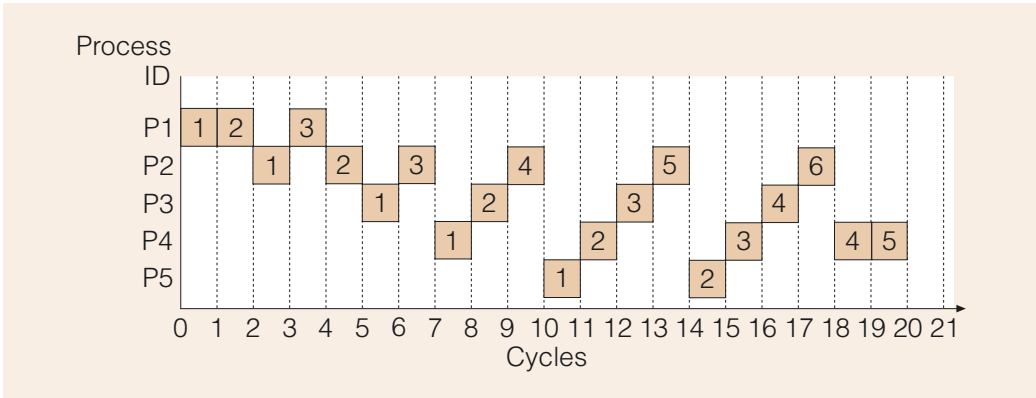


Figure 11 Round robin execution of processes in Table 2 with quantum = 1

Each process executes for as many quanta as before, but over the course of more cycles than before, due to pre-emption and having to wait for another turn in the queue of processes. This leads to the process metrics given in Table 4.

Table 4 Round robin timings for some processes

Process ID	Arrival time (a)	Service time (s)	Finish time (f, as shown in Figure 11)	Turnaround (q = f – a)	Normalised turnaround (n)
P1	0	3	4	4	1.33
P2	2	6	18	16	2.67
P3	4	4	17	13	3.25
P4	6	5	20	14	2.80
P5	8	2	15	7	3.50

Now the mean normalised turnaround value is 2.71, while the median is 2.80, somewhat worse than before, but the maximum value is now 3.5, nearly halving the FIFO case.

The key parameter in this system is the quantum, which is typically from about a hundredth of a second to about one second long. A small quantum tends to make the system more responsive, while a long one tends to provide greater throughput because there is less context switching. However, when the quantum is too small, too much time may be spent in context switching. The quantum must be large with respect to the time required to context switch to avoid excessive overhead in context switching.

Although we have only shown a quantum of one cycle, more generally a process that requires I/O may only run for part of its quantum before it is blocked on an I/O requirement as compared to a CPU-bound process. At this point it will wait for its I/O in a blocked state and cede the rest of its quantum. Later it will become runnable again, being returned to the back of the queue. CPU-bound processes, in contrast, will tend to consume their full quantum on any given turn. So this policy results in relatively good response times for CPU-bound processes as well as processes requiring only short amounts of CPU time.

A variation on this scheme is to maintain a round robin queue for each process priority and dispatch from these in order of priority, so that a lower-priority process only runs when there are no queued higher-priority processes.

There are many other schemes for scheduling policies which we do not consider further here, but the interested reader may refer to, for example, Tanenbaum (2001), Bacon (1998) or Deitel et al. (2003). However, it is important to remember that a programmer does not normally know what scheduling algorithm is employed by the operating

system, and programs that rely on assumptions about what scheduling system is employed are unlikely to be portable.

SAQ 8

- (a) Explain how the timer interrupt mechanism together with the process scheduler assists in achieving multitasking on a single processor.
- (b) Why is a pre-emptive scheduling policy preferred in interactive systems?

ANSWER.....

- (a) A timer interrupt can be used by a scheduler to decide points in time at which to apply a selection function, that is, to pre-empt a running process. This may result in a decision to allow a different process to run from the one currently running. At this point the scheduler can initiate a context switch to make the processor deal with another task until the next switch should take place.
- (b) To provide greater fairness and responsiveness to processes requiring interaction.

Exercise 2

- (a) Suppose that a context switch takes 10 milliseconds and that the quantum is set to 10 milliseconds. Why is this not an efficient value for the quantum? What would be better?
- (b) In the round robin scheme, what is the effect of using a very large quantum?
- (c) Suppose that the quantum is 1 millisecond and the time to context switch is 10 milliseconds. What is the effect?
- (d) Comparing the round robin scheduling algorithm and the FIFO algorithm, under what circumstances are these more or less equivalent?

Discussion.....

- (a) In 20 milliseconds, the most the CPU can do is one quantum and one context switch. This means it spends a large proportion (about half) of its time context switching. It may be better to have a larger quantum in this case. For example, if the quantum is set at 30 milliseconds, then in 40 milliseconds the CPU might have done one context switch and spent 75% of its time on executing a process.
- (b) The responsiveness of the system will degrade as the size of the quantum gets larger and individual processes keep a hold on the CPU for longer, starving other processes of time to the point where it becomes noticeable that some processes are not responding.
- (c) As the size of the quantum drops below the length of time to perform a context switch, the CPU spends most of its time context switching. In this case, in 100 milliseconds the CPU will have done 9 context switches, and executed processes for only 10 milliseconds – it will thus have spent 90% of its time on context switching.
- (d) When the quantum becomes large with respect to service times, most jobs will run to completion without pre-emption, so the round robin algorithm will approximate to a FIFO algorithm. For the example in the text, when the quantum is greater than or equal to 6, the two policies are the same.

6

Threads and processes

During the previous sections, where we have been studying the concept of a process, you may have become aware that there are two potentially independent concepts wrapped up in the idea of a process: control of some resources and execution of some code.

It is therefore possible to say that a process is both of the following.

- ▶ *A unit of resource ownership.* We saw that a process is allocated a virtual address space (that is, one that can be recreated as physical addresses in main memory when required) to hold its data and will be assigned other resources such as files or devices as required.
- ▶ *An item that can be scheduled for execution.* The process records its path through its instructions (in the form of a program counter) so that it has an independent record of its progress. In this sense, a process forms a unit of dispatching; that is, it can be scheduled.

Here we look at how these two concepts can be managed separately using the concept of a thread, and at how a Java Virtual Machine (JVM) handles threads.

6.1 Threads

We introduce here a new term for the case when there may be multiple independent paths of execution within a single unit of resource ownership. Each path of execution is called a **thread**, which is short for **thread of execution**.

For example, within one program you might want to monitor the keyboard for a key being pressed by a user and, at the same time, track the movement of the mouse and simultaneously repaint the screen. Each one of these tasks can be realised as a single thread in a program. Many scientific or 'number-crunching' applications can also take advantage of threading, though not all algorithms are amenable to breaking into threads in this way, or would benefit greatly in execution time from parallelisation.

Even where program performance does not benefit, threads are often a more intuitive way of organising code to do a job. For example, a game can run a thread to handle each player, or a server a thread to handle each client. A new thread can be created for a new player or new client, and the same code reused. The single-threaded alternative might be relatively complex, for example it would require working through a list of clients or players in turn, attempting to ensure that none is ignored for too long.

A thread, like a process, can be dispatched by a (thread) scheduler, so you can also consider a thread to be a unit of dispatching, or unit of execution. However, several threads may share the same resources. In particular, threads will share the same address space, whereas processes do not normally do so. We say that a process is **multithreaded** when it contains more than one thread of execution.

Threading is both an approach to sharing the resources of a computer, enabling a program to make best use of the resources available, and also a programming tool to help break down work into easily managed parts.

6.2 The Java Virtual Machine (JVM) and operating system support for threading

The mapping between coded threads and operating system processes is platform-dependent.

When we run a JVM we create a process. Java processes will have multiple threads, created by the user and the Java Runtime. Bear in mind that it is possible to run more than one virtual machine at a time and that each virtual machine appears to the operating system as a separate process.

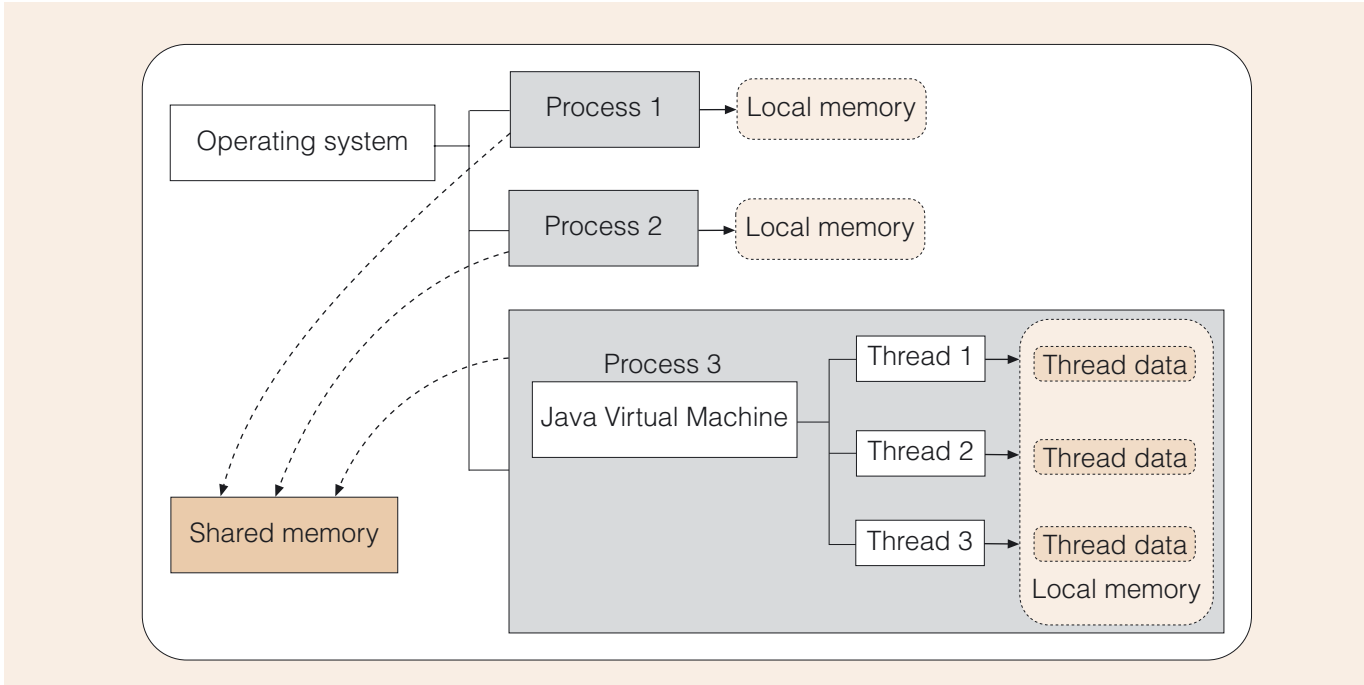


Figure 12 Allocation of memory in multithreading

Figure 12 is similar to Figure 4 with three processes acquiring their own local memory from a shared memory pool controlled by the operating system. However, Process 3 now represents a JVM. Three threads are illustrated sharing the local memory acquired by the JVM, each of which has its own thread data within the same local memory address space.

Thus threads may communicate information using shared memory without recourse to any special mechanisms. This kind of communication can be simpler than that required for processes to communicate with one another.

An operating system that supports threads is referred to as a **multithreaded operating system**. Analogously to the ideas of pre-emptive and cooperative multitasking, we can talk about pre-emptive or cooperative multithreading. In **pre-emptive multithreading** systems, the operating system can determine when a context switch between threads needs to occur, whereas **cooperative multithreading** relies on threads giving up control of a processor to allow other threads time to run.

In a multithreaded Java program, the threads may be scheduled by the JVM's own thread scheduler (depending on the JVM implementation). Looking at Figure 12 again, it is easy to see that things can get quite complex because, from the operating system's point of view, the JVM is a process. Thus the JVM itself gets scheduled like any other process, and when the JVM is executing it will make decisions as to which thread gets to run. Note that even if the operating system itself does not support multithreading, the JVM may. In the next section we will study in detail how the JVM thread scheduler works.

Context switching between threads in the same process requires relatively little overhead, compared to context switching between processes, since shared memory

will be preserved during the switch. For this reason, threads are sometimes called **lightweight processes**, or ‘cheap’ processes.

We can therefore identify three types of process placement as:

- ▶ lightweight processes, sharing the same address space, in which case they are normally referred to as threads (Java threads are an example);
- ▶ processes that run in separate address spaces on the same machine (applications running on a PC are an example);
- ▶ processes that run in separate address spaces on different machines (for example client and server applications operating over a network).

SAQ 9

- (a) What is the difference between a thread and a process?
- (b) What is a multithreaded operating system?

ANSWER.....

- (a) Threads (lightweight processes) are a unit of dispatching but not of resource management. Processes are a unit of both dispatching and resource management. In particular, threads share an address space (memory) through which they can communicate ‘cheaply’ as compared to processes. This is because when there is a context switch it is unnecessary to change resource information, thereby reducing the time taken for the switch. A heavyweight process has its own copy of all resources and so context switches have a larger overhead.
- (b) A multithreaded operating system is one that supports the execution of lightweight processes.

6.3 Handling of memory in Java programs

There are traditionally two kinds of memory allocated by runtime systems, known as **stack** and **heap**. These names refer to the data structures underlying the management of the memory. A stack is an area of memory that is last-in, first-out (LIFO). A heap is a ‘pile’ of memory, in that parts of it can be allocated and deallocated without reference to any particular structure or order.

In Java the stack is used to store **activation frames** of methods, which contain sufficient information to resume execution of those methods when their nested or recursive method calls (if any) complete. An activation frame contains a return address, arguments passed to the method and space for local variables, as well as a pointer to the previous frame, for example.

The size of any primitive and reference variables recorded is fixed and can therefore be determined in advance (statically).

In contrast, objects are less well defined – the object could be any size. Since it is not possible to know the size of objects in advance, they are allocated on a heap structure as the memory required is determined, at runtime. Thus we can say that anything that has a reference in a Java program is allocated on the heap. When objects are no longer needed, they are garbage collected and their memory is returned to the heap.

Garbage collection of an object takes place some time after there are no references to it remaining in scope.



Activity 2.2

Stack trace.

The predictability of sizes of references and primitives versus the unknown size of objects is one of the key reasons for the separation of storage into stack and heap. Of course, both the heap and the stack contents grow and shrink dynamically, according to the sequence of methods called within the program.

Consider the following code fragment representing a `main` method:

```
public static void main(String args[])
{
    System.out.print(1);
}
```

This causes the activation frame for `main` and then for `print` to be added to a stack. The `print` method in turn shows the following code:

```
print(int i)
{
    write(String.valueOf(i));
}
```

This results in activation frames for `write` and `valueOf` being added to the stack. This is illustrated in Table 5, where the lowest method is the one first called, and subsequent activation frames are piled on top of it.

Table 5 A stack of activation frames

valueOf activation frame
write activation frame
print activation frame
main activation frame

The last method to be called is also the first to be removed from the stack – its results can be returned to the method represented by the activation frame underneath it that will then have ‘risen’ to the top of the stack.

Eventually the `main` method completes and the program terminates.

When there are multiple threads, each thread may produce its own stack of this sort. The virtual machine maintains a ‘master copy’ of any variables. From time to time, under the control of various rules, threads copy information to or from the master copy. This has consequences for how you manage threads that interact with the same methods, as discussed in *Unit 3*.

You will read more about the JVM and its handling of memory in *Unit 4*.

7

Programming threads in Java

Java provides a great deal of support for the writing of multithreaded applications. Each task can be made into a thread, and on a multiprocessor computer a number of threads can truly run at the same time. Both the virtual machine and user programs can create a number of threads.

Just as processes are the dynamic forms of programs, threads are not the Java code itself, but rather they are a dynamic form of executing that code. If you are running a single JVM, these threads all exist in the same runtime environment and are not different programs. However, each thread may be between its start and finish points.

This section first provides a quick review of how to create Java threads. We then move on to discuss the model for Java thread states and the way this model is used by the JVM thread scheduler.

7.1 Creating threads

In Java we trace program execution from the start of the `main` method, and a single user thread begins running when this method is invoked. There will also be a number of virtual machine threads running at the same time. You can then create extra user threads in addition to these.

Just as all Java programs begin at their `main` method, all threads begin executing at their `run` methods.

There are two ways to program threads: by inheriting from the `Thread` class or by implementing the Java interface `Runnable`. In either case, a `run` method must be defined by the programmer.

Threads follow paths of execution through your Java methods. Like a process, a `Thread` object contains 'housekeeping' information, which includes a program counter and pointers to data and instructions, for example.

We will now look at examples of creating threads by extending `Thread` and implementing `Runnable`.

Extending the `Thread` class

The simplest way to create a thread is to define a class extending the `Thread` class and override the inherited `run` method.

The `Thread` class is in the `java.lang` package, which is imported automatically.

A class called `WhoAmI` inheriting from the `Thread` class is shown below.

```
public class WhoAmI extends Thread
{
    String who;

    public WhoAmI (String n)
    {
        who = n;
    }

    /*override the run method*/
    public void run()
    {
        for (int i = 0; i < 10 ; i++)
        {
            System.out.println("Thread " + who);
        }
    }
}
```

Once you have defined your thread class you can create an instance of it. Starting the thread is achieved by invoking the `start` method, which is also inherited from `Thread`. The `start` method registers the thread with the thread scheduler, which can run the thread when it determines that the time is right. The `run` method is then invoked for you.

So we might create and use two instances of this class as follows:

```
public class ThreadTester
{
    public static void main(String[] args)
    {
        WhoAmI thread1 = new WhoAmI ("Joe");
        thread1.start();

        // or create and start an anonymous thread
        new WhoAmI("Bloggs").start();
    }
}
```

The class produces three user threads: the original thread (executing the method `main`) that invoked the `start` methods and the two new threads, each of which now executes its `run` method.

Note that if you had invoked the `run` method directly, you would not have created a new thread. Instead the `run` method would execute under the control of the currently executing thread, just like a normal method call.

We passed each instance of `WhoAmI` a different `who` value so that we can identify which thread is running at any particular time. Although you might expect that the code in the `thread1` instance runs first, followed by the anonymous thread, in fact the output is determined by when the thread scheduler decides to run each thread, and output may be interleaved.

```
Thread Joe
Thread Bloggs
Thread Joe
Thread Bloggs
Thread Joe
Thread Bloggs
Thread Bloggs
Thread Joe
Thread Joe
...
```

The output is platform-dependent and may vary from one run to another. The number of times each line is printed reflects the time the thread is allowed to run before a context switch occurs. It is quite possible that a thread completes its run within that time, so that output is not interleaved.

SAQ 10

Assuming the same `WhoAmI` class shown earlier, how many threads does the following code create within the Java Virtual Machine?

```
public class ThreadTester2
{
    public static void main(String[] args)
    {
        WhoAmI thread1 = new WhoAmI ("Joe");
        WhoAmI thread2 = new WhoAmI ("Sue");
        thread2.run();
        thread1.start();
        new WhoAmI ("Bloggs").start();
    }
}
```

ANSWER.....

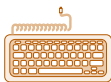
This code creates three threads: the thread started by invoking `main`, the thread created by invoking `start` on the `Thread` object referenced by `thread1`, and the anonymous thread we passed the value "Bloggs" to. Invoking `run` on the `Thread` object referenced by `thread2` does not create a new thread. The `run` method would be executed within the `main` user thread, like a normal method call, and would be completed before `thread1.start()` is invoked.

In addition, the Java Runtime will have created other threads, such as the garbage collection thread.

Implementing the `Runnable` interface

An alternative approach to defining code for a thread is by using the `Runnable` interface. The Java API recommends implementing `Runnable` unless the class is intended to have an inheritance relationship with `Thread`, for example, because it overrides a thread method other than `run`, or because it needs to inherit one of the `Thread` methods, such as the `interrupt` method.

Recall also that Java does not allow multiple inheritance, thus when a class already explicitly extends another, the only way it can be run as a thread is for it to implement `Runnable`.



Activity 2.3
Creating threads.

The `Thread` class has a constructor that takes a reference to a `Runnable` object as an argument, so it is possible to create a thread from an object of a class that implements `Runnable`.

The `Runnable` interface contains one method with the header `public void run()` (like that of the `run` method in the `Thread` class). Any class implementing `Runnable` correctly can have its `run` method executed as a thread. For example, we can create a similar class to our previous example, called `WhoAmI2`, which could subclass another class at the same time:

```
public class WhoAmI2 extends SomeClass implements Runnable
{
    String who;

    public WhoAmI2 (String n)
    {
        who = n;
    }

    // implement the run method in some way
    public void run()
    {
        for (int i = 0; i < 10 ; i++)
        {
            System.out.println(who + who.length());
        }
    }
}
```

In this case we implement, not override, `run`.

We have written a slightly different `run` method, but otherwise the main difference is that the class implements the `Runnable` interface.

Creating an instance of the class implementing the `Runnable` interface does not create a new thread.

To create a thread from this object, you need to first create an instance of `WhoAmI2` and then use it as an argument to the appropriate `Thread` class constructor. This will create a thread with a `start` method, which can be used as before to activate the thread. The code below illustrates this procedure to create three threads that execute the code in the `run` method of `WhoAmI2`.

```
public class ThreadTesterRun
{
    public static void main(String[] args)
    {
        // create WhoAmI2 objects
        WhoAmI2 runnable1 = new WhoAmI2("Joe");
        WhoAmI2 runnable2 = new WhoAmI2("Bloggs");

        //Create a Thread with an identifier t1 using a runnable
        Thread t1 = new Thread(runnable1);
        t1.start();

        //or create an anonymous thread using a runnable
        new Thread(runnable2).start();

        // create an anonymous thread from anonymous runnable
        new Thread(new WhoAmI2("Jane")).start();
    }
}
```


Each `run` method is run in a thread, and so output may be interleaved as before:

```
Joe3
Joe3
Bloggs6
Joe3
Joe3
Joe3
Bloggs6
Jane4
Joe3
...
```

As before, the output is platform-dependent and may vary from one run to another.

User threads and daemon threads

A thread that is used to execute a background job can be marked as a **daemon thread** to indicate this. Often such threads are in endless loops and so may not terminate of their own accord. You can set a thread as a daemon thread by using the `setDaemon` method, passing the argument `true`, which must be done before `start` is executed on the thread. This indicates to the virtual machine that it does not have to wait for this thread to terminate before the JVM exits.

The JVM garbage collector is an example of a daemon thread.

Normally a thread is created as a non-daemon or **user thread**. As long as at least one of these is running, the virtual machine should continue operating.

SAQ 11

- What are the two ways to define a thread?
- When a thread is executing, does it always keep executing until it gets to the end of its code without interruption?
- How do you tell the system that you want to set a thread running?
- When would you typically want to define a thread by implementing `Runnable` rather than extending `Thread`?
- How do you define, create and start a thread using `Runnable`?
- When would you want to create a daemon thread?
- What kind of thread is created by invoking `main`?

ANSWER.....

- A thread can be defined by extending the `Thread` class or by implementing the `Runnable` interface.
- No, a thread is unlikely to keep executing until it comes to the end of its code. In all likelihood the thread will only have access to the CPU for a short part of its execution before it is stopped and another thread is given an opportunity to run.
- To set a thread running after creating an instance of a threaded class, the method `start` can be invoked.
- There are many occasions when a threaded class will be inheriting from another class, for example `JFrame`. As Java does not allow multiple inheritance, if such classes require threading, the only way to do this is by implementing the `Runnable` interface. However, it is generally best to implement `Runnable` when you do not require an inheritance relationship with `Thread`.

- (e) To define, create and start a thread using `Runnable`, first implement the interface `Runnable`, which requires implementing the `run` method required by the interface. Then create an instance of the `Runnable` class and create a new `Thread` object using the `Thread` constructor that takes a `Runnable` object as an argument. The `Thread` object thus created can be started in the usual way, by invoking the `start` method.
- (f) You may want to create a daemon thread when the thread is intended to run without intervention, or when it is safe for the virtual machine to exit whether or not the thread has completed running. This is often the case for threads that do background work.
- (g) Invoking `main` creates a user (non-daemon) thread.

7.2 Thread states

Like processes, threads may also be categorised according to the state they are in – this can be used for scheduling purposes.

In this subsection we will concentrate on thread states as reported by the Java Virtual Machine. As Java code is running in a virtual machine, these states are the virtual machine classification of thread state, and the underlying mapping to operating system threads is an unknown.

The possible thread states and the transitions between them are illustrated in Figure 13.

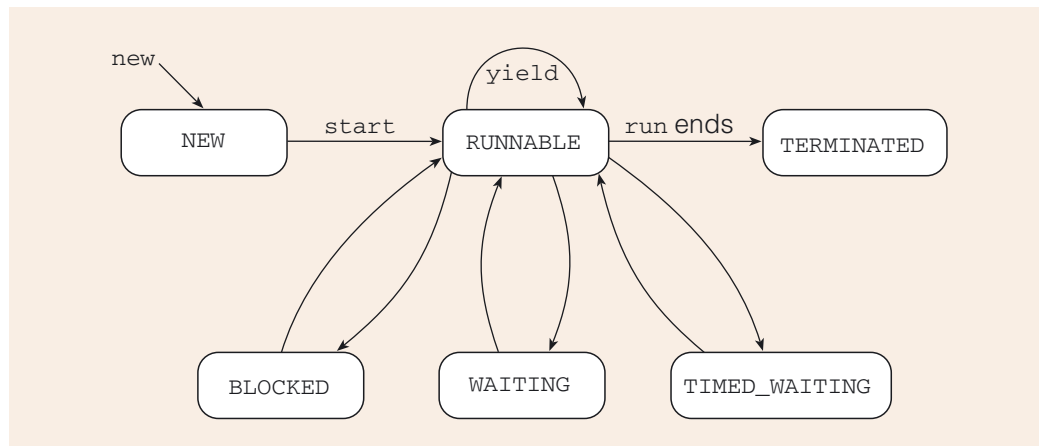
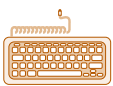


Figure 13 The six-state model for Java threads in the JVM

From Java 1.5 onwards a method called `getState` has been provided to allow the current state of a thread to be identified (it will be a value of a type called `Thread.State`). The states in `Thread.State` are defined as follows.

- ▶ **NEW**
A thread that has not yet started is in this state.
- ▶ **RUNNABLE**
A thread that is runnable in the JVM is in this state. Note that this means that the thread could be running, or it could be waiting for the operating system to provide a resource, such as CPU time. There may be many **RUNNABLE** threads at any given time. A method called `yield` is discussed in Subsection 7.4 that allows a thread to give way to other threads temporarily while remaining **RUNNABLE**.
- ▶ **BLOCKED**
A thread that is waiting to enter a controlled region of code (called `synchronized`) is in this state. The detail of the **BLOCKED** state is deferred until *Unit 3*.



Activity 2.4

Thread states in Java.

- ▶ `WAITING`
A thread that is waiting indefinitely for another thread to perform a particular action is in this state. The detail of this state is deferred until *Unit 3*.
- ▶ `TIMED_WAITING`
A thread that is waiting for another thread to perform an action for up to a specified waiting time is in this state. The thread may also simply be ‘sleeping’ for a while.
- ▶ `TERMINATED`
A thread whose `run` method has completed is in this state.

Movement from one state to another is governed by the operating system and other external factors, as well as by which methods are invoked on the threaded objects.

You may like to think of each state as having a circular arrow pointing to itself, as the thread remains in the state unless some event occurs.

An interesting point about this set of states is that there is no ‘running’ state, as only the underlying operating system can actually cause a thread to run. In the JVM specification the state `RUNNABLE` covers both the case that a thread is actually running and the case where the thread is waiting for CPU time to run.

The thread lifecycle

Let us then summarise the normal lifecycle of a thread in the JVM.

Initially a thread object is constructed by creating an object from `Thread` or one of its subclasses, creating a thread in the `NEW` state.

The `start` method moves the thread into the `RUNNABLE` state. This does not mean that it will get a chance to run immediately. For this to be the case, the JVM must be selected by the operating system process scheduler as the process to run.

When a particular thread is chosen for the first time, its `run` method will be invoked and the thread can begin to execute the code. (From the operating system point of view, it will be in the running state.) If it finishes executing it moves into the `TERMINATED` state. However, the code in all likelihood will not reach its conclusion before other events intervene. Assuming a pre-emptive scheduler (the normal case), the thread may run out of its allotted CPU time (quantum), or *alternatively*, it may invoke the `yield` method (see Subsection 7.4) to give way to other threads of equal or higher priority. Both scenarios will cause the thread to go back into the `RUNNABLE` state, where it can, at some point, be chosen by the Java Runtime system to be run again. Once it is back in the running state from the operating system point of view, it can carry on with the execution of its `run` method at the point where it left off.

The thread then continues to be selected by the JVM scheduler from time to time if it is `RUNNABLE`. A thread may be paused for some time waiting for an event to occur (although this does not include I/O events, as far as the JVM is concerned, as these are under the control of the operating system).

When a thread enters the `TERMINATED` state it cannot be restarted, but an associated object may still be accessible and its methods used as usual. A new object is required in order to start another thread of the same kind.

7.3 Thread priorities

Java threads are given a priority between 1 and 10 expressed as static constants in the `Thread` class, with `MIN_PRIORITY` and `MAX_PRIORITY` having the values 1 and 10.

A thread inherits the priority of its creator. The `main` method (from which, ultimately, all user threads come into existence) has by default a priority of 5. This is implemented in the `Thread` class as the constant `NORM_PRIORITY`.

The priority of threads can be dynamically changed by invoking the `setPriority` method of the thread.

The scheduler can use priorities to decide which thread to run, but policies may vary from one virtual machine to another, and the mapping of threads to the underlying platform may vary. Therefore any program logic that relies on priorities set by the programmer will not be platform-independent. In practice you will not often find yourself needing to set priorities.

Mars Pathfinder

The 1997 'Pathfinder' mission produced exciting images of the surface of Mars for a few days before the spacecraft software began resetting itself. What went wrong?

The real-time software used pre-emptive priority-based scheduling of threads. A low-priority process gathered meteorological data, and a high-priority system process handled the flow of data through a bus, while a medium-priority process handled a communication task between the two.

It was possible for an interrupt to occur under heavy loads that would cause the communication task to be scheduled at the same time as the bus had to wait for the meteorological data task to complete. Since the communication task had a higher priority than the meteorological task, it would take preference. After some time, a watchdog task would notice that the meteorological task was not proceeding, and reset the system.

Fortunately, the problem could be corrected remotely by ensuring that the priority of the meteorological task increased after it received rights to use the bus, so that it could not be pre-empted by the communication task.

7.4 Methods controlling thread behaviour

In this subsection we will discuss some useful methods to control thread behaviour. More advanced methods relating to thread behaviour are presented in *Unit 3*.

The `sleep` method

The `sleep` method is a class-wide (static) method of the `Thread` class invoked to indicate that the currently executing thread can be moved from the `RUNNABLE` state to `TIMED_WAITING`.

Invoking `Thread.sleep(n)`, where `n` is a number in milliseconds, causes the currently executing thread to enter the `TIMED_WAITING` state for at least `n` milliseconds, after which it is returned to the `RUNNABLE` state.

As with other static methods, `sleep` can be invoked on object references, but it is poor style to do so and it may make you think that you are suspending a thread in the object

you referenced. The reality is that you are always suspending the thread *from* which sleep was invoked.

There is also a method that allows you to specify a more precise sleep period, `Thread.sleep(long milliseconds, int nanosecs)`; however, this is a higher precision than can reasonably be expected to be measured most of the time.

The `yield` method

The `yield` method is a static method of the `Thread` class invoked to indicate that the currently running thread can allow other threads a chance to proceed.

The Java API indicates that invoking `yield` (which takes no arguments and returns no value) causes the currently executing thread object to temporarily pause and allow other threads to execute. However, the thread can remain `RUNNABLE` if no other threads are ready to run, or the JVM scheduler may detect that no other threads are `RUNNABLE` and ignore the call. In addition, the scheduler may decide that any other `RUNNABLE` threads are not important enough (based on their priority) and select the thread that invoked `yield` to execute again.

However, the point is that `yield` can cause the thread to give up the CPU, which may be necessary in cooperative multitasking systems (where pre-emption by the operating system does not happen) to force a thread to take a break.

Brogden says of this:

The language specification does not require any particular method for thread scheduling. A computationally intensive program that appears to run fine without using `yield` calls on systems using time-slicing scheduling may show unacceptable delays on systems that use a different method of scheduling.

Brogden (1999)

So, in the interests of platform-independence it may be best to be polite to other threads and invoke `yield`, which is rather like saying ‘after you’, from time to time if the thread is likely to take a long time to do its work. However, you need to be sure that it is safe for other threads to proceed, and that there is no harm in pausing the current thread.

The `join` method

The `join` method is an instance method and so is invoked on a `Thread` object reference. This causes the currently executing thread to wait until the thread on which `join` is invoked has terminated.

For example, if a piece of code starts a thread referenced by `t`, and then invokes `t.join()`, this causes the currently executing thread to wait until the thread referenced by `t` has completed.

Uses of this include retrieving a value from a thread object before continuing, or to ensure that some work is done before continuing.

Returning to our `WhoAmI` class, consider the following `main` method:

```
public static void main(String[] args)
{
    // create and start a thread with handle thread1
    WhoAmI thread1 = new WhoAmI("Joe");
    thread1.start();
    thread1.join(); // wait for thread1 to end

    // create and start an anonymous thread
    new WhoAmI("Bloggs").start();
}
```

In this case, `main` starts and creates `thread1` then `thread1` runs to completion. Only then does the anonymous thread get started.

As for the `sleep` method there are two variations of the `join` method that allow you to specify a maximum time to wait for a thread. Again the methods allow the wait time to be specified in milliseconds or a combination of milliseconds and nanoseconds. This may be useful if you can only wait for so long before having to continue. It is, however, unlikely that you will want to specify how long to wait for a thread at the nanosecond level.

The `currentThread` method

The `currentThread` method is a static `Thread` method that returns a reference to the currently executing thread, and so can be used to obtain information about it, such as its name (using `getName`), its priority (using `getPriority`), its string representation (using `toString`), or whether it has started and not yet terminated (`isAlive`).

For example, we could write

```
public static void main(String[] args)
{
    Thread t = Thread.currentThread();
    System.out.println(t.getName());
}
```

This will print out the name of the `main` thread.



Activity 2.5

Java thread behaviour.

SAQ 12

Under what circumstances should a Java thread relinquish a processor? How could it do so?

ANSWER.....

A thread may initiate a context switch when it is waiting for an event or simply to allow other threads a chance to run, by invoking the `yield` method, or the `sleep` method.

7.5 The `InterruptedException`

Finally, it is worth mentioning that a thread may throw an `InterruptedException`, which is caused by the `interrupt` method being invoked on the thread when it is paused for some reason, for example, when it is sleeping. This is a checked exception, and so the compiler will require you to deal with it. Typically, you may want to throw this exception upwards, so that it is raised at a higher level, but you can also display information about it when catching it, as with any other exception. This is of most interest when you are dealing with communicating processes, which is discussed in more detail in *Unit 3*. However, we will not consider the various implications of a thread raising this exception, which are beyond the scope of this course.

7.6 GUIs and threads

Since threads share memory it becomes the programmer's rather than the operating system's responsibility to mediate between them. There is one Java issue that is worth some mention here.

GUI components in Java are maintained by their own thread called the **event-dispatching thread**. This thread handles rendering of components and user-interaction with them, for example it reacts to events such as a user clicking on a menu or using a keyboard to type in a text box. This thread also recognises events such as a window becoming covered or uncovered, so that it can redraw any components that need redrawing.

To avoid problems with threads interfering with each other, it is important that code affecting the appearance of GUI components runs in the event-dispatching thread. Invoking such code in an arbitrary way (from another thread) can lead to hard to trace bugs.

The following are some guidelines for working with GUIs.

- 1 For graphical operations, the code updating the component should be performed from within the `paint` method. The `paint` method is not invoked directly, but rather the program requests that `paint` is invoked using the `repaint` method. The `paint` method is then invoked at a convenient moment, from the correct thread. The `paint` method will also be automatically invoked when certain window events occur, such as a window being moved, or part of a window being covered by another.
- 2 For GUI event-handling (for example button clicks, radio-button selection, dropdown list selection and text input), code updating the component should be performed via the `actionPerformed` method. This method is invoked automatically when such events occur and runs in the correct thread.

If you are interacting with variables relating to the drawing of a GUI component, you need to be aware that the event-dispatching thread may also access them in order to redraw the component, and that therefore there may be issues with sharing data between threads.

In general, if you cannot use `paint` or `actionPerformed`, you should request that the GUI updating code is executed in the event-dispatching thread. Two utility methods `invokeLater` and `invokeAndWait` will allow you to do this. We do not consider these further here, but you can read about these methods in the API for the `SwingUtilities` class if you are interested in digging deeper. This class also includes a method `isEventDispatchThread` to allow you to determine whether code is running in the event-dispatching thread.

8

Summary

In this unit we have looked at the basis of multitasking and parallel processing and how programs interact with the platforms on which they run.

In order to manage the running of several tasks concurrently, processes are used to describe a task under execution. Processes may be in various states and their context may be stored and restored, allowing context switching to take place.

Hardware platforms may provide multiple processors enabling true parallel execution, or there may be pseudo-parallelism where a processor shares its time between multiple processes, via time-slicing. This is supported by the interrupt mechanism.

Interrupts are one of two techniques for a program to determine whether or not an event has occurred, with the other being polling. Polling requires that code repeatedly assesses some condition to determine whether the event has occurred, while an interrupt is generated as a result of an event occurring, and can be used to signal to other code that the event has occurred. A buffer may be used to smooth out communications between processes or devices and to prevent one process from having to wait for another.

When multiple processes need to be executed, they may end up having to share the CPU. It is the job of a scheduler to apply a selection function, either pre-emptively or non-pre-emptively, to determine which process should run when and hence when a context switch is required. Scheduling policies aim to provide fairness to processes, to utilise resources fully, and to keep systems responsive to users. In cooperative multitasking systems only processes themselves can initiate a context switch.

We have briefly discussed three possible scheduling policies (FIFO, SJF and round robin) and introduced some simple metrics by which such policies might be compared. One factor that may be employed in scheduling policies is the priority assigned to each process.

At the application level a programmer may be able to specify parts of programs that can be run in parallel using threads, which are lightweight processes that share a memory space. Threads, like processes, are also classified by their state. We have introduced the thread states in Java and the possible transitions between them, as well as some of the methods that affect thread state.

At the virtual-machine level, threads may also be subject to scheduling or this job may be done by the operating system. A virtual machine is itself a process and subject to scheduling by the underlying platform.

Java provides a great deal of support for multithreading and we have reviewed how to create threads, as well as exploring some methods that can be used to control thread behaviour. A programmer is responsible for controlling threads' access to the memory they share. This affects GUIs in Java and we have suggested some techniques for handling this interaction.

Unit 3 picks up the issue of thread interaction in more detail, while *Unit 4* will discuss more about the relationship between the JVM and the underlying platform.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ discuss the relationship between code, process, virtual machine, operating system and hardware;
- ▶ explain what a process is and how processes can be paused and restarted;
- ▶ identify different states that processes can be in and how process states can be modelled to achieve multitasking;
- ▶ explain the roles of scheduling software, discuss some scheduling algorithms and evaluate some metrics relating to scheduling;
- ▶ distinguish between a process and a thread;
- ▶ discuss polling and interrupts as methods for communication between processes (or threads);
- ▶ write code to create threads in Java and use the `sleep`, `yield` and `join` methods on threads in Java;
- ▶ discuss threading issues with graphical interfaces in Java.

Glossary

activation frames A data structure storing information about a method call in Java that allows the method to resume after being placed on a stack.

batch process A process that does not require user interaction.

blocked process A process that is unable to proceed because it is waiting for some event to occur.

buffer An area of memory used for communication between processes or devices.

busy-waiting Repeatedly checking and rechecking some condition in a loop until it becomes true.

cache An area of high-speed memory local to the CPU holding recently accessed data.

compilation The process by which source code is translated into machine code in one operation.

context A record of the condition of a process at a point in time, that is, the internal state of the process (not to be confused with process state, which is a categorisation of the process at a higher level).

context switch The saving of the context of one process and the loading of the context of another.

context switching The act of performing a context switch.

cooperative multitasking Multitasking that is dependent on processes voluntarily ceding way to each other from time to time.

cooperative multithreading Multithreading that is dependent on threads voluntarily ceding way to each other from time to time.

daemon thread A thread that runs without interaction and which the JVM does not have to wait for when exiting.

data width The amount of data that a device can handle at a time.

device controller A piece of hardware that can interact with a device independently of a system processor.

device driver A piece of software used to communicate with a hardware device.

dispatcher See **short-term scheduler**.

event-dispatching thread A Java thread that can be used to respond to events in GUIs.

faults A class of interrupt caused by hardware failure.

fetch-execute cycle The cycle by which the next instruction to be executed is retrieved and completed by a CPU.

heap A dynamically allocated area of memory used to store objects in the JVM.

interpreted (programming language) A programming language where source code is translated to machine code in stages rather than all at once.

interrupt A mechanism by which a device or software can signal that an event has occurred.

interrupt handler A software routine responsible for managing interrupts.

interrupt rate limiting A technique employed by hardware to prevent a flood of interrupts being created.

interrupt storm The condition under which more interrupts are generated than can be dealt with.

I/O interrupts The class of interrupts concerned with input and output events.

Java Runtime Environment (JRE) An implementation of the Java Virtual Machine specification, providing an environment supporting the loading and running of Java programs and support from library code.

Java Virtual Machine (JVM) A specification for the Java Runtime Environment.

lightweight process See **thread**.

long-term scheduler A process scheduler that controls the admission of new processes to a system to achieve a desirable mix of runnable processes.

machine code Platform-dependent, CPU-specific, low-level code.

medium-term scheduler A process scheduler that manages the swapping of processes to and from secondary storage.

multitasking Execution of several tasks at the same time, whether by parallel or concurrent processing techniques.

multithreaded Containing more than one thread.

multithreaded operating system An operating system that supports multithreaded processes.

non-pre-emptive multitasking A process scheduling policy allowing a process to run to completion before applying a selection function.

normalised turnaround time The value q/s , where q is the turnaround time and s is the service time.

null process A process executed by the scheduler when no other processes are Runnable; can be used to perform housekeeping tasks or to simplify a scheduling algorithm.

persistent storage Storage that does not require power (cf. **volatile**) such as a disk drive.

platform The combination of hardware and software underlying a program's execution.

polling The technique of repeatedly checking for an event.

pre-emptive multitasking A process scheduling policy that may require a process to give up the CPU before it would ordinarily have done so.

pre-emptive multithreading A thread scheduling policy that may require a thread to give up the CPU before it has completed running.

priority A number used to indicate the importance of a process, thread or interrupt.

privileged instructions Machine code instructions that may only be executed by a processor in supervisor mode.

process A program under execution.

process descriptor A data structure containing sufficient information about a process to allow it to be paused and restarted.

process identifier A unique value (ID) distinguishing one process from another.

process scheduler A general term for software that manages process state transitions according to a scheduling policy.

process state A high-level categorisation of a process in terms of what it may do next, for example, Runnable or Blocked.

process table A data structure used by an operating system to store information about processes.

program counter A pointer to the next instruction to be executed in a program.

program interrupts A class of interrupt generated by software due to an exception.

quantum A time during which a process may run under a pre-emptive scheduling policy.

registers High-speed memory, local to the CPU, used to execute instructions.

responsiveness An indication of whether a system is acceptably fast in its response to a request for action or information (the definition of what is acceptable depends on the context).

runnable process A process that may be selected to be run; same as a ready process.

running process A process that is being executed by the CPU.

runtime environment The software environment supporting loading, linking to libraries and execution of programs.

scheduling policy A policy contributing to process state change at any given time and in particular determining which process is allowed to run.

selection function A function used by a scheduler to select which process is allowed to run.

service time The time a process would run if it had sole use of the CPU.

short-term scheduler A process scheduler concerned mainly with which process is allowed to run next.

stack A last-in, first-out (LIFO) data structure, used to store activation records for Java threads.

supervisor call interrupt An interrupt used to signal that supervisor instructions are to be executed.

supervisor mode The mode a CPU enters to execute supervisor instructions.

suspended process A process that is stored on a disk; same as swapped out.

swapped out See **suspended process**.

swapping The action of moving a process from volatile to persistent storage.

system bus A hardware channel for data transmission between device controllers and CPU.

task A sequence of instructions treated as an element of work. A task may be represented by a method in a Java program, for example, but the term is more widely applied and can refer to elements of work at a low (processor) or high (application) level.

thread A lightweight process; a thread of execution.

thread of execution A path of execution within a process.

throughput The number of processes completed per unit of time by a CPU.

timer interrupts A class of interrupt used to keep track of the passage of time; used by schedulers to pre-empt processes.

turnaround The time interval between the submission of a process and its completion.

user mode The CPU mode when it is executing ordinary (not supervisor) instructions.

user thread A thread created by a user; a non-daemon thread.

virtual memory External memory; persistent storage used to supplement main memory.

volatile storage Storage that requires power in order to retain any data stored in it, for example RAM, cache and registers.

References

Bacon, J. (1998) *Concurrent Systems* (2nd edn), Addison-Wesley.

Brogden, B. (1999) *Java 2, Certification Insider Press*.

Deitel, H.M., Deitel P.J., and Choffnes, D.R. (2003) *Operating Systems* (3rd edn), Prentice Hall.

Oaks, S. and Wong, H. (2004) *Java Threads* (3rd edn), O'Reilly.

Stallings, W. (1991) *Operating Systems* (2nd edn), Macmillan.

Tanenbaum, A. (2001) *Modern Operating Systems* (2nd edn), Prentice Hall.

Index

- A
 - activation frame 35
- B
 - batch
 - process 7, 26, 28
 - processing 7, 27
 - buffer 14, 16
 - busy-waiting 14–15
- C
 - cache 9
 - compilation 8
 - context 18
 - context switching 15, 18–19, 31–34
 - overhead 34
 - CPU 10
 - CPU bound 27, 30–31
- D
 - data width 9
 - device 12
 - calls 12
 - controllers 10
 - drivers 12
 - interface 12
 - disk storage 10
 - dispatcher 26
- E
 - efficiency 27
 - event 14, 18
 - exception 15
- F
 - fairness 7, 27, 32
 - faults 15
 - fetch–execute cycle 11
 - Floating Point Unit 10
- G
 - grid computing 7
 - GUI components 47
- GUI-related methods
 - actionPerformed 47
 - isEventDispatchThread 47
 - paint 47
 - repaint 47
- H
 - hardware
 - devices 12
 - interfaces 10
- I
 - idle process 26
 - illegal instruction 15
 - interface 11
 - device–operating system 12
 - software 11
 - interpreted code 8
 - interrupt 14–16, 26
 - handlers 15
 - I/O 15
 - prioritisation 16
 - program 15
 - rate limiting 16
 - storm 16
 - supervisor call 15
 - timer 15, 25, 32
 - I/O 22, 26–27, 31, 43
- J
 - Java Virtual Machine (JVM) 12, 34, 42
 - JFrame class 41
- K
 - kernel mode 8
- L
 - library 12
 - calls 12
- M
 - machine code 8
 - main method 37
 - Mars Pathfinder 44
- memory
 - heap 35
 - local 19, 34
 - persistent 10
 - shared 19, 34
 - stack 35
 - virtual 10
 - volatile 10
- memory allocation 35
 - LIFO (last-in, first-out) 35
- metrics 27
- multiple inheritance 39, 41
- multiprocessor system 19, 37
- multitasking 6, 32
- multitasking operating system
 - cooperative 7, 45
 - non-pre-emptive 7
 - pre-emptive 7
- multithreaded operating system 34
 - cooperative 34
 - pre-emptive 34
- multithreaded process 33
- N
 - normalised turnaround time 28
 - null process 26
- O
 - operating system 6, 8, 12, 26
- P
 - platform 6
 - dependent 8, 39
 - independent 45
 - polling 14
 - priority 25
 - privileged instructions 8, 15

- process 6
 - blocked 21
 - context 18
 - descriptors 18, 21
 - dispatch 21
 - identifiers 18
 - lightweight 35
 - placement 35
 - priorities 27
 - queues 20
 - ready 21
 - resources 21–22, 26–27
 - runnable 21, 31
 - tables 19–20
- process model
 - five-state 21
 - seven-state 22
 - two-state 20
- process operation
 - activate 23
 - admit 21, 23
 - create 21
 - dispatch 20–21
 - event occurred 21
 - event wait 21
 - release 20
 - sleeping 23
 - suspend 22
 - timeout 20
- process scheduler 20, 22, 24, 43
 - desirable characteristics 27
 - JVM 30
 - long-term 26
 - medium-term 26
 - short-term 26
- process state 18, 20, 23, 26
 - Blocked 21–23, 26, 31
 - Blocked Suspended 22–23
 - Exiting 21–22
 - New 21, 23, 26
 - Not Running 20
 - Runnable 21–23, 25–26
 - Runnable Suspended 22–23
 - Running 20–21, 26
 - Suspended 26
- program counter 11, 18, 33
- pseudo-parallelism 7
- Q
 - quantum 20, 22, 26, 30–32
- R
 - RAM 9
 - register(s) 9
 - instruction 11
 - responsiveness 7–8, 25, 27, 32
 - Runnable interface 37, 39
 - runtime environment 8, 12
 - Java Runtime Environment (JRE) 12, 34, 43
- S
 - scheduling
 - algorithm 31
 - priority-based 44
 - scheduling policy 26
 - FIFO (first-in, first-out) 28, 31–32
 - pre-emptive 27
 - round robin 30, 32
 - SJF (shortest job first) 30
 - selection function 25
 - selection function (application of)
 - non-pre-emptive 25
 - pre-emptive 25
 - service time 28
 - shared resources 7
 - signalling 15
 - storage *see* memory
 - supervisor mode 8, 15
 - suspended 22
 - swapped out 22
 - swapping 22, 27
 - SwingUtilities class 47
 - synchronized keyword 42
 - system bus 10
 - system calls 12
- T
 - tasks 6
 - Thread class 37
 - thread 33, 35
 - anonymous 38
 - daemon 41
 - event-dispatching 47
 - lifecycle of 43
 - of execution 33
 - scheduler 33
 - user 37, 41
 - virtual machine 37
 - thread states
 - BLOCKED 42
 - NEW 42
 - RUNNABLE 42–44
 - TERMINATED 43
 - TIMED_WAITING 43–44
 - WAITING 43
 - thread-related method
 - currentThread 46
 - getState 42
 - interrupt 39
 - join 45
 - run 37–38, 43
 - setDaemon 41
 - setPriority 44
 - sleep 44
 - start 38, 43
 - yield 42–43, 45
 - Thread.State type 42
 - throughput 25, 27
 - time-slicing 20, 22, 26, 45
 - timing out 22
 - turnaround 27–28
- U
 - unit
 - of dispatching 33, 35
 - of resource ownership 33
 - user
 - mode 8
 - processes 15
- V
 - virtual address space 33